# Biomni: A General-Purpose Biomedical AI Agent

Kexin Huang[1,*,‡], Serena Zhang[1,*], Hanchen Wang[1,2,*], Yuanhao Qu[3,4,5,*], Yingzhou Lu[5,*], Yusuf Roohani[1,6], Ryan Li[1], Lin Qiu[7], Gavin Li[1], Junze Zhang[3,5], Di Yin[3,5], Shruti Marwaha[8], Jennefer N. Carter[8], Xin Zhou[5], Matthew Wheeler[8], Jonathan A. Bernstein[9], Mengdi Wang[10], Peng He[11], Jingtian Zhou[6], Michael Snyder[5], Le Cong[3,5], Aviv Regev[2], and Jure Leskovec[1,‡]

[1]Department of Computer Science, Stanford University School of Engineering, Stanford, CA, USA

[2]Research and Early Development, Genentech, South San Francisco, CA, USA

[3]Department of Pathology, Stanford University School of Medicine, Stanford, CA, USA

[4]Cancer Biology Program, Stanford University School of Medicine, Stanford, CA, USA

[5]Department of Genetics, Stanford University School of Medicine, Stanford, CA, USA

[6]Arc Institute, Palo Alto, CA, USA

[7]Paul G. Allen School of Computer Science and Engineering, University of Washington, WA, USA

[8]Department of Medicine, Stanford University School of Medicine, Stanford, CA, USA

[9]Department of Pediatrics, Stanford University School of Medicine, Stanford, CA, USA

[10]Department of Electrical and Computer Engineering, Princeton University, Princeton, NJ, USA

[11]Department of Pathology, University of California San Francisco, San Francisco, CA, USA

∗Equal contribution.

‡Corresponding authors. Email: kexinh@cs.stanford.edu, jure@cs.stanford.edu

# Abstract

**Biomedical research underpins progress in our understanding of human health and disease, drug discovery, and clinical care. However, with the growth of complex lab experiments, large datasets, many analytical tools, and expansive literature, biomedical research is increasingly constrained by repetitive and fragmented workflows that slow discovery and limit innovation, underscoring the need for a fundamentally new way to scale scientific expertise. Here, we introduce Biomni, a general-purpose biomedical AI agent designed to autonomously execute a wide spectrum of research tasks across diverse biomedical subfields. To systematically map the biomedical action space, Biomni first employs an action discovery agent to create the first unified agentic environment – mining essential tools, databases, and protocols from tens of thousands of publications across 25 biomedical domains. Built on this foundation, Biomni features a generalist agentic architecture that integrates large language model (LLM) reasoning with retrieval-augmented planning and code-based execution, enabling it to dynamically compose and carry out complex biomedical workflows – entirely without relying on predefined templates or rigid task flows. Systematic benchmarking demonstrates that Biomni achieves strong generalization across heterogeneous biomedical tasks – including causal gene prioritization, drug repurposing, rare disease diagnosis, microbiome analysis, and molecular cloning – without any task-specific prompt tuning. Real-world case studies further showcase Biomni's ability to interpret complex, multi-modal biomedical datasets and autonomously generate experimentally testable protocols. Biomni envisions a future where virtual AI biologists operate alongside and augment human scientists to dramatically enhance research productivity, clinical insight, and healthcare. Biomni is ready to use at https://biomni.stanford.edu, and we invite scientists to explore its capabilities, stress-test its limits, and co-create the next era of biomedical discoveries.**

# 1 Introduction

Biomedical research is a key pillar of modern science and medicine, driving discoveries in disease mechanisms, diagnostics, and therapeutics[1–4]. Yet, with the growth in large-scale experiments, data, tools, and literature, progress is increasingly slowed by fragmented, complex workflows that require specialized tools, exhaustive literature reviews, intricate experimental design, and careful statistical modeling[5,6]. A vast volume of valuable biomedical data sits underutilized[7], many sophisticated analyses are not conducted, and many connections for past knowledge and literature are not made, not for lack of significance, but because the demand for expert researchers far exceeds the supply. This mismatch between data abundance and limited human bandwidth highlights an urgent need for a fundamentally new approach – one that can effectively scale expertise, streamline workflows, and unlock the full potential of biomedical research.

Recent advances in Artificial Intelligence (AI) have created a paradigm shift, opening the possibility for fundamentally reshaping biomedical research[8]. AI agents have dramatically reshaped fields such as software engineering[9], law[10], material science[11] and healthcare[12] by automating repetitive tasks, enhancing productivity, and enabling breakthroughs that were previously unimaginable. Given these developments, the question emerges: *Can we build a virtual AI biomedical scientist?* Such a virtual scientist would autonomously tackle diverse biomedical research tasks spanning multiple subfields, unlocking extensive capabilities and fostering novel insights through interdisciplinary integration – an achievement that can radically augment human biologists limited by specialized expertise. Capable of efficiently managing thousands of concurrent tasks, this virtual scientist could dramatically enhance human productivity and accelerate the pace of biomedical discovery.

Previous approaches have largely relied on specialist agentic workflows tailored to narrow biomedical tasks[13–19], which restricts their capacity to move fluidly and generalize across the full spectrum of biomedical domains, as needed to answer key research questions. Enabling an AI agent to handle a broad range of biomedical tasks introduces substantial technical challenges – most notably, the need to tightly couple advanced reasoning[20] with the ability to execute highly specialized biomedical actions[21]. Although LLM-based reasoning has seen significant advancements[22], such LLMs need access to an environment that explicitly defines the biomedical

action space, which is inherently diverse, domain-specific, and complex. Moreover, a truly capable system requires an agentic architecture that can natively interact with this biomedical environment – autonomously selecting and composing actions, using its reasoning capabilities to plan and execute diverse tasks without relying on rigid, pre-defined workflows.

Here we present Biomni, a generalist biomedical AI agent purpose-built to automate and advance biomedical research across a wide range of subfields. Acting as a virtual AI biologist, Biomni autonomously formulates novel, testable hypotheses, performs complex bioinformatics analyses, and designs rigorous experimental protocols. To enable this capability, we first constructed a unified and comprehensive biomedical action space by systematically analyzing tens of thousands of biomedical research papers spanning 25 distinct subfields, curated from major bio-literature repositories. From this foundation, we developed an LLM-powered action discovery agent capable of reading papers and extracting key tasks, tools, and databases essential to driving biomedical discoveries. These elements are then selected and implemented into Biomni-E1, the foundational environment that defines the biomedical action space for agentic interaction. Biomni-E1 includes 150 specialized biomedical tools, 105 software packages, and 59 databases. We then designed Biomni-A1, a general-purpose agent architecture capable of flexibly executing a broad spectrum of biomedical tasks by using tools and datasets provided by Biomni E1. Given a user query, the agent first uses a retrieval system to identify the most relevant tools, databases, and software needed. It then applies LLM-based reasoning and domain expertise to generate a detailed, step-by-step plan. Each step is expressed through executable code, enabling precise and flexible compositions of biomedical actions – an essential feature given the domain's reliance on highly specialized tools and data resources. Unlike traditional function-calling methods, this approach supports the dynamic and complex nature of biomedical workflows. This integrated system allows Biomni not only to solve challenging, large-scale biomedical problems with efficiency, but also to generalize to novel tasks across previously unseen areas of biomedical research.

Rigorous benchmarking demonstrates Biomni's outstanding performance across established biomedical Q&A benchmarks, and robust generalization performance in eight challenging, realistic scenarios never encountered during development. Additionally, we highlight Biomni's practical capabilities through three impactful case studies: (1) analyzing 458 files of wearable sensor data to generate novel insights; (2) rapidly performing comprehensive bioinformatics analyses on mas-

sive raw datasets, such as single-cell RNA-seq and ATAC-seq data, to generate novel insights and hypotheses; (3) autonomously designing laboratory protocols to assist wet-lab researchers. With Biomni, we introduce the first generation of a scalable, general-purpose biomedical AI agent, setting the stage for an era where virtual AI biologists work alongside human researchers to dramatically accelerate biomedical discovery from basic research to translation.

## 2 Results

**Overview of Biomni.** Biomni is a general-purpose biomedical AI agent comprising two main components: Biomni-E1, a foundational biomedical environment with a unified action space, and Biomni-A1, an intelligent agent designed to utilize this environment effectively.

Curating a unified biomedical action space is challenging due to its inherent complexity and vastness. We systematically address this by employing an AI-driven approach (Figure 1a). Specifically, we leveraged the 25 subject categories defined by bioRxiv, selecting the 100 most recent publications per category. An action discovery LLM agent processed each paper sequentially, extracting essential tasks, tools, databases, and software necessary to replicate or generate the described research. This comprehensive set of resources constitutes the essential actions required to perform a large set of biological research tasks.

We then curated Biomni-E1, an environment for a biomedical AI agent to perform a wide range of actions (Figure 1b). Identified tools were rigorously verified by human experts, along with corresponding test cases. These tools (Supplementary Table 1-Table 18) were specifically chosen for their non-trivial nature, encompassing complex code, domain-specific know-how, or specialized AI models. Recognizing the inherent flexibility required by biological software, which cannot always be simplified into static functions, we constructed an execution environment pre-installed with 105 widely-used biological software packages (Supplementary Table 23-30), supporting Python, R, and Bash scripts. For database integration, we categorized resources into two distinct groups. The first group consists of massive relational databases accessible via web APIs (e.g., PDB, OpenTarget, ClinVar) (Supplementary Table 19-20). Rather than creating numerous individual retrieval tools, we implemented a unified function per database. Each function accepts natural language queries and internally employs an LLM to parse database schemas and generate executable queries dynamically. Databases without web interfaces were downloaded into a data

lake and preprocessed locally into structured pandas DataFrames for seamless integration with the agent, for a total of 59 databases in Biomni-E1 (Supplementary Table 21-22). In summary, Biomni-E1 is the first environment for biomedical AI agent and includes 150 specialized biomedical tools, 105 software, and 59 databases.

To build a general-purpose agent capable of tackling diverse biomedical tasks, we require a specialized agentic architecture – one that avoids hardcoding workflows for each individual task. This led to the development of Biomni-A1, which incorporates several core innovations critical for operating across the biomedical research landscape. First, we introduce an LLM-based tool selection mechanism designed to navigate the complexity and specialization of biomedical tools, dynamically retrieving a tailored subset of resources based on the user's goal. Second, recognizing that biomedical tasks often require rich procedural logic, Biomni-A1 uses code as a universal action interface – allowing it to compose and execute complex workflows involving loops, parallelization, and conditional logic. Crucially, this approach also enables the agent to interleave calls to software, tools, databases, and raw data operations that do not conform to predefined function signatures-supporting flexible and dynamic integration of heterogeneous resources. Third, the agent adopts an adaptive planning strategy: it formulates an initial plan grounded in biomedical knowledge and iteratively refines it throughout execution, enabling responsive, context-aware behavior. Together, these innovations enable Biomni-A1 to generalize to previously unseen tasks and domains, dynamically composing intelligent actions and interfacing with software, data, and tools in a way that embodies generalist biomedical intelligence (Figure 1c).

**Biomni excels on general biomedical knowledge and reasoning benchmarks.** We evaluated Biomni on three challenging multiple-choice benchmarks of general biomedical knowledge and reasoning: Humanity's Last Exam (HLE)[23] and LAB-Bench[24], which includes two key subtasks – DbQA (Database Question Answering) and SeqQA (Sequence Question Answering) (Figure 2a). These tasks span tool use, symbolic reasoning, and structured biological information retrieval – core competencies for any robust biomedical AI agent. To isolate the impact of tool access and agent design, we compared Biomni against six strong baselines (details in Supplementary Notes A).

For LAB-Bench, a 45-question development set was used to refine tool and database inter-

faces, while the final evaluation was conducted on 315 held-out test questions, with performance averaged across three independent runs. We only curated a representative 12.5% subset of the full benchmark due to API cost constraints. In DbQA, which requires structured querying over biological databases, Biomni achieved 74.4% accuracy – matching expert human performance (74.7%) and outperformed all baselines, including the coding agent (ReAct+Code, 40.8%). In SeqQA, which involves reasoning over DNA and protein sequences, Biomni achieved 81.9% accuracy, again exceeding human-level performance (78.8%).

To test true generalization of biomedical knowledge and reasoning *without any development set*, we also evaluated Biomni on a 52-question subset of HLE spanning 14 biomedical subfields – from molecular biology to physiology. Biomni achieved 17.3% accuracy, significantly outperforming the base LLM (6.0%), coding agent (12.8%), and literature agent (12.2%). These results demonstrate Biomni's ability to generalize across unfamiliar, open-ended biomedical domains without any task-specific adaptation. Additional ablation results are shown in Supplementary Figures 1-2. Performances across each subfield are reported in the Supplementary Figure 3.

**Biomni generalizes to new, real-world biomedical tasks across diverse subfields.** To evaluate generalization in realistic research tasks, we curated eight new biomedical benchmarks spanning genetics, genomics, microbiology, pharmacology, and clinical medicine (Figure 2b). Each task was framed to reflect a common, well-defined, but complex real-world biomedical research goal, including: (1) Variant prioritization: Identify the most likely causal variant from a list of potential variants for a trait, requiring reasoning about regulatory functions in non-coding regions. (2) GWAS causal gene detection: Select the most likely causal gene within a locus, demanding fine-grained locus-level inference. (3) CRISPR perturbation screen design: Construct gene panels to maximize post-perturbation effect across a large ($\sim$20,000 genes) search space. (4) Rare disease diagnosis: Map patient phenotypes and genetic findings to rare disease diagnosis. (5) Drug repurposing: Given a rare disease and a list of candidate drugs, select the best therapeutic match. (6) Single-cell RNA-seq annotation: Assign accurate cell-type labels to individual cell profiles across tissues, species, and platforms. (7) Microbiome disease-taxa analysis: Perform statistical association tests on microbiome datasets to uncover disease-relevant taxa. (8) Patient gene prioritization: Given an individual patient's genetic profile and phenotype description, identify the most plausi-

ble causal gene. We benchmarked Biomni without prompt engineering or task-specific fine-tuning against three baselines: (1) a base LLM (Claude Sonnet 3.7) without tool use, (2) a coding agent with direct function calls and code execution (ReAct+Code), and (3) Biomni-ReAct, an ablation of Biomni that replaces code-based planning with ReAct-style chaining. The complete benchmark constructions are described in Methods, with detailed performance comparisons in Supplementary Notes B.

Across all tasks, Biomni outperformed the base LLM by an average relative performance gain of 402.3%, the coding agent by 43.0%, and its own ablated variant Biomni-ReAct by 20.4%. These findings highlight the importance of code-centric planning and environment grounding, enabling Biomni to compose precise, flexible, and context-aware actions. For each benchmark, we further analyzed the execution trajectories, identifying commonly invoked tools, software, and datasets, as detailed in Supplementary Figures 6-16. These trajectories provide insight into the complexity and structure of agent behavior across tasks. On average, Biomni executes between 6 and 24 distinct steps per task, involving combinations of 0-4 specialized tools, 1-8 software packages, and 0-3 unique data lake items. The agent interleaves data extraction, search/retrieval, reasoning, and computational analyses (Supplementary Figure 8) – reflecting a workflow pattern that mirrors how human scientists alternate between retrieving knowledge and generating new insights. Resource usage varies by task type: information synthesis tasks, such as CRISPR perturbation screen design and GWAS causal gene identification, rely heavily on database queries (e.g., KEGG, Reactome) and literature search (e.g., PubMed, Google), whereas bioinformatics analysis tasks like microbiome profiling and single-cell annotation involve minimal database use but extensive code execution with software libraries such as scanpy.

**Biomni jointly analyzes 458 wearable sensor files to generate physiological hypotheses.** To evaluate Biomni's performance in real-world biomedical workflows, we invited scientists to apply it directly to their own research questions. In this case study, a researcher used Biomni to analyze 458 Excel files containing months-long wearable sensor data (continuous glucose monitoring (CGM) and body temperature) from 30 participants. The data were highly heterogeneous: file formats varied, annotations were inconsistent, and participants exhibited substantial variability (Figure 3a). The researcher posed an open-ended question: Can we uncover biologically meaning-

ful thermogenic patterns?

Biomni autonomously generated and executed a 10-step analysis pipeline (Figure 3b), inferring meal events from glucose spikes, extracting pre/post meal temperature windows, normalizing across individuals, and synthesizing population-level trends. Crucially, after completing the pipeline, the agent delivered a structured, human-readable report summarizing its key findings (Supplementary Notes D). It identified a consistent postprandial thermogenic response, with an average temperature rise of 2.19°C (median: 1.10°CC) and a wide range across individuals (-0.11°CC to 15.56°CC). Some participants showed rapid, pronounced spikes within 30 minutes of eating, while others had delayed or muted responses – indicating divergent metabolic phenotypes (Figure 3c,d). These insights were not manually curated or extracted by a human; the agent performed the entire analysis end-to-end and surfaced the results as a concise narrative highlighting patterns that would otherwise being ignored in raw data.

In a parallel workflow, the scientist requested Biomni to analyze 227 nights of wearable-recorded sleep data across 10 participants. Biomni computed averages for duration, efficiency, latency, and sleep stage composition, derived a composite sleep quality score, and conducted chronobiological analyses. The agent delivered a structured summary to the user (Supplementary Notes D, Supplementary Figure 4), including personalized sleep profiles and timing insights, without human post hoc synthesis. Biomni uncovered several novel insights: sleep efficiency consistently peaked mid-week (on Wednesdays) and declined on Sundays, suggesting a potential behavioral pattern tied to pre-Monday stress or weekend-induced disruptions. Another important finding was that consistent sleep timing correlated more strongly with higher sleep quality than total sleep duration, highlighting the critical role of circadian regularity in maintaining restorative sleep.

The scientist then tasked Biomni with analyzing multi-omics data (652 lipidomic, 731 metabolomic, and 1,470 proteomic features), jointly with the CGM data. Biomni conducted cross-omics correlation analysis, applied hierarchical clustering to uncover biologically coherent feature groups, and performed unsupervised PCA to link CGM signals to molecular pathways. It automatically generated interpretable outputs – trajectory plots, heatmaps, boxplots, PCA biplots, and cluster maps – empowering rapid insight generation from complex multimodal datasets (Supplementary Notes D, Supplementary Figure 5). Significant correlations among lipids, metabolites, and proteins revealed

tightly interlinked regulatory pathways, underscoring the systems-level nature of metabolic regulation. Notably, several identified biomarkers showed consistent patterns across samples and exhibited high connectivity within correlation networks. Across all cases, the scientist noted that Biomni accelerated the path from messy real-world data to testable hypotheses, supporting applications in sleep optimization, metabolic research, and precision health.

**Biomni automates complex multi-omics analysis to decipher transcriptional regulation of skeletal lineages.** To test whether Biomni could generalize to complex omics workflows, a scientist used it to analyze a recently published multi-omics dataset of the developing human skeleton [25]. This dataset comprises 336,162 single-nucleus RNA-Seq (snRNA) and ATAC-seq (snATAC-Seq), paired with spatial transcriptomics data collected from human embryos between 5-11 weeks post-conception (Figure3e). While the original study emphasized developmental trajectories and disease mechanisms, the scientist was interested in exploring gene regulatory mechanisms across emerging skeletal cell types – a technically demanding task typically requiring extensive bioinformatics support.

The scientist asked Biomni to investigate transcriptional regulation across skeletal lineages using a detailed instruction (Supplementary Notes E). The system autonomously planned and executed a ten-stage analysis pipeline: (1) loading and exploring all datasets, (2) preparing RNA-seq data for analysis, (3) configuring pySCENIC to retrieve motifs, (4) running GRNBoost2 to infer gene regulatory networks, (5) pruning networks using cisTarget, (6) calculating regulon activity with AUCell, (7) extracting accessibility data from ATAC-seq, (8) filtering predicted targets using ATAC-seq accessibility, (9) analyzing activity patterns across cell types, developmental stages, and anatomical regions, and (10) summarizing findings and preparing a report to the scientist. It enabled Biomni to predict transcription factor-target gene links and filter regulons based on motif enrichment and chromatin accessibility correlations (Figure 3f). The full run, completed in just over five hours, handled real-time execution issues (e.g., variable name mismatches) by subsampling and debugging locally. Throughout, Biomni maintained all intermediate outputs – code, figures, and logs – organized in a reproducible folder structure for validation and inspection. The agent summarized all the analysis and generated a report describing the analysis and key findings (Supplementary Notes E).

In its final gene regulatory network (GRN) analysis (Figure 3h), Biomni re-capitulated known regulatory relationships between key osteogenic transcription factors such as RUNX2 and HHIP, confirming how they are regulated by a shared set of anti-osteogenic transcription factors including TWIST1, LMX1B, and ALX4 [25]. These findings align with author's report [25] about the balanced regulation needed for proper bone formation and suture patency. Furthermore, Biomni also uncovered several unreported TFs, including AUTS2, ZFHX3, and PBX1, showed unexpectedly high regulatory activity across multiple skeletal cell types. Although PBX1 is a well-established skeletal regulator [26] and ZFHX3/AUTS2 have only limited or indirect skeletal reports (in mouse [27] or zebrafish [28]), their broad activity here suggests under-appreciated roles across diverse skeletal lineages. Biomni reported that these novel regulators were particularly active in osteoblasts, preosteoblasts, and various chondrocyte populations, suggesting they play important but previously unrecognized roles in the transcriptional control of skeletal cell fate determination during human embryonic development. Finally, Figure 3g-h reveals how Biomni's visualizations effectively captured both temporal dynamics of regulator activity and cell-type-specific variations in key regulons like RUNX2. This demonstrates how Biomni enables researchers to autonomously perform complex multi-omics analysis and rapidly generate testable hypotheses without specialized programming expertise.

**Biomni designs wet-lab validated experimental protocol for cloning.** To evaluate Biomni's ability to support real-world experimental design, we focused on a core task in molecular biology: cloning. This process is central to countless workflows in research and biotechnology and requires complex reasoning, from designing high-fidelity primers to choosing the right assembly method and validating constructs. While general-purpose LLMs have struggled to perform such tasks due to limited domain knowledge and tool access [24], Biomni integrates LLM reasoning with dynamic tool execution, enabling expert-level performance in molecular biology tasks.

To rigorously evaluate this task, we first collaborated with an expert group of gene-editing researchers to design an open-ended cloning benchmark and expert user study (Figure 4a). Our benchmark consisted of 10 realistic, representative cloning tasks covering Golden Gate, Gibson, Gateway, and restriction cloning – each with options including single-fragment vs. pooled assembly. The benchmark also included essential validation steps, such as designing Sanger sequencing

primers and analyzing restriction digests. We posed these tasks to four entities: an LLM (Claude 3.7), Biomni, a human trainee (Stanford Biology Master with previous experience in cloning), and a senior human expert (Stanford Genetics PostDoc with 5+ years of cloning experience). Each was asked to generate a complete, end-to-end protocol along with the final cloned plasmid map. Blinded expert reviewers assessed the outputs. Biomni produced protocols and designs that matched the human expert in accuracy and completeness – often providing comparable levels of detail and anticipating the same edge cases. In contrast, the human trainee's submissions were frequently incomplete or suboptimal, reflecting the experience gap typical in early-stage researchers. Remarkably, Biomni completed all tasks autonomously in a fraction of the time taken by the expert.

To further validate Biomni in a real-world setting, a scientist assigned it a practical cloning task: cloning a guide RNA targeting the human B2M gene into the lentiCRISPR v2 Blast construct (Figure 4b). Biomni successfully executed the task through a comprehensive workflow (Figure 4c). First, it analyzed the plasmid structure using annotation and pattern search tools to identify key features necessary for cloning. It then designed three Cas9 sgRNAs targeting B2M using specialized knockout sgRNA design tools. For the cloning process, Biomni generated forward and reverse oligos with BsmBI overhangs to enable directional insertion of the sgRNA sequence. It produced detailed protocols (Figure 4d) for oligo annealing, double-stranded DNA formation, and Golden Gate cloning into the target vector. Biomni also provided complete bacterial transformation instructions, including heat-shock steps and antibiotic selection. For quality control, it designed a U6 promoter sequencing primer to verify sgRNA insertion and simulated the Golden Gate assembly to produce the final plasmid map.

The scientist followed Biomni's protocol exactly to perform the wet-lab experiment (Figure 4e). Colonies appeared on the plate the next day; two were cultured, miniprepped, and sequenced using the Biomni-designed primers – both showing perfect alignment. This case illustrates how scientists can rely on Biomni to autonomously design complex molecular biology experiments with accuracy comparable to human experts, but in a fraction of the time.

**User-friendly interface to empower scientists to generate biomedical discoveries.** To bring the power of Biomni into the hands of every scientist, we built an intuitive graphical interface – avail-

able at https://biomni.stanford.edu – to help transform the way researchers interact with biomedical data and tools. This seamless platform enables users to submit natural language queries and receive results powered by the full capabilities of Biomni's agentic system. Whether designing complex cloning experiments, querying multi-omics databases, or generating hypotheses from wearable data, scientists can now access the intelligence of a general-purpose biomedical AI agent without writing a single line of code. The interface is designed for rapid iteration, real-time feedback, and visual traceability, allowing users to explore intermediate steps, inspect tool usage, and validate results interactively. By closing the gap between biomedical intent and execution, Biomni opens a new era of accessible, automated, and scalable scientific discovery. An example of this interface is shown in Supplementary Figure 17.

# 3 Discussion

Biomni marks a major step forward in biomedical research, demonstrating robust generalization across diverse subfields and laying the groundwork for AI agents as integral collaborators in scientific discovery. Its zero-shot performance across complex tasks – including those in genetics, genomics, microbiology, immunology, pharmacology, and clinical medicine – underscores its potential to boost research productivity, accelerate discovery, and broaden access to advanced biomedical analyses.

By automating complex, labor-intensive workflows, which normally require both expert knowledge and coding skills, Biomni enables researchers to redirect their efforts toward creative hypothesis generation, experimental innovation, and cross-disciplinary collaboration. This shift holds profound implications. In the conext of target and drug discovery for biopharma, Biomni can autonomously prioritize targets, design perturbation screens, or repurpose drugs – offering a path to faster, more cost-effective reasearch. In clinical application settings, its capabilities in gene prioritization and rare disease diagnosis point to more accurate, personalized insights and streamlined diagnostics. For consumer health, Biomni's integration of wearable data and multi-omics analyses envisions real-time, individualized health monitoring and intervention.

Nonetheless, several limitations remain. While Biomni's unified environment spans a wide range of biomedical tools and databases, the evaluated tasks represent only a subset of the field, and key domains remain unexplored. In addition, in the action discovery agent, our decision to pri-

oritize the most recent literature makes the agent appear timely, but risks overlooking foundational concepts and techniques that have faded from current discourse despite their enduring relevance. The future versions should encapsulate a larger coverage of publications when defining the environment. Moreover, although Biomni approaches human-level performance in tasks like database querying, sequence analysis, and molecular cloning, it still struggles in areas requiring nuanced clinical judgment, novel experimental reasoning, analytical inventions, or deep biological thinking and synthesis. No system yet captures the full scope of human biomedical expertise. As reflected in our benchmarks, Biomni has not achieved expert-level performance across all task categories. We expect continued improvements as foundation models evolve and the agentic environment expands, as well as thanks to human experts and trainees deploying Biomni to facilitate or augment their work.

These limitations open promising directions for future development. Training biomedical reasoning agents with reinforcement learning could enable continuous self-improvement in planning and execution. Integrating multimodal data – text, images, and structured inputs – may further deepen reasoning capabilities. Equipping Biomni to autonomously discover and incorporate new tools and databases, as well as incorporate more historical methods (which may have high utility but can be easily forgotten by human users), would ensure adaptability and long-term relevance.

Looking ahead, Biomni and its successors could become foundational infrastructure in an AI-powered biomedical ecosystem, working seamlessly with human experts to unlock novel insights into health and disease. This hybrid partnership may radically reshape biomedical research – automating hypothesis generation, scaling discovery pipelines, and enabling medical innovation to proceed at unprecedented speed and scope. General-purpose agents like Biomni could not only accelerate breakthroughs but redefine the future of scientific inquiry itself.

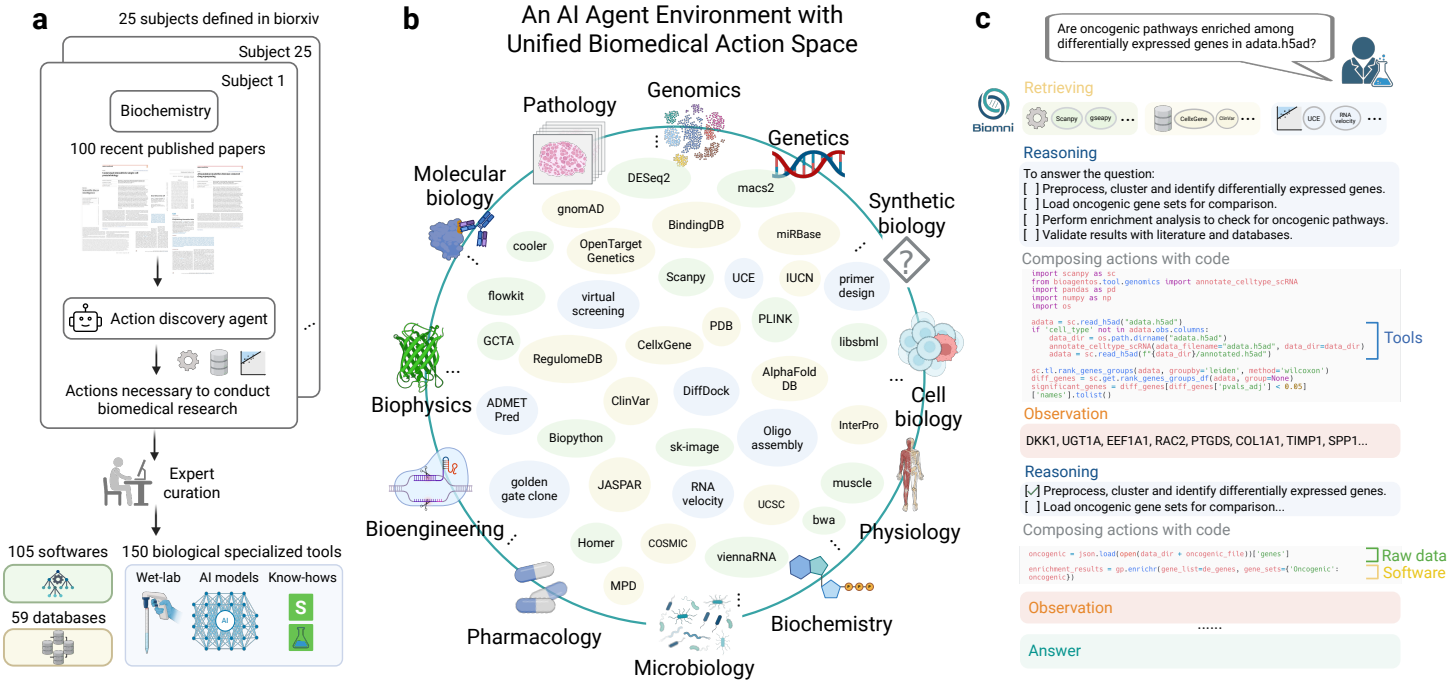



**Figure 1:** Overview of the unified biomedical action space and agent environment in Biomni. (a) Workflow for systematically curating the unified biomedical action space. Actions necessary to conduct biomedical research were extracted from 2,500 recent bioRxiv publications across 25 biomedical subfields using an AI-driven discovery agent. Extracted actions were rigorously validated and curated by human experts, resulting in the integration of 105 biomedical software tools, 150 specialized biological tools (including wet-lab protocols, AI-driven predictive models, and domain-specific know-how), and 59 comprehensive biomedical databases. (b) Illustration of the unified biomedical action space spanning diverse biomedical subfields such as genetics, genomics, synthetic biology, cell biology, physiology, microbiology, pharmacology, bioengineering, biophysics, molecular biology, and pathology. Representative tools and databases integrated into Biomni's environment are shown, highlighting its general-purpose capabilities. (c) Example workflow demonstrating Biomni's reasoning and action composition process to autonomously answer a complex biological question. Biomni retrieves relevant tools based on the user's query, formulates a structured reasoning plan, and composes executable code to perform comprehensive bioinformatics analyses, iteratively refining its reasoning based on observations until converging on a final, precise answer.




**Figure 2:** Zero-shot generalization of Biomni across diverse realistic biomedical tasks. (a) Biomni is superior to 6 baselines in Q&A multiple choice benchmarks that broadly evaluate the model's capability across biomedical fields. (b) Biomni demonstrates robust zero-shot performance across eight previously unseen, real-world biomedical scenarios spanning multiple biomedical subfields, without any task-specific fine-tuning or prompt engineering. Evaluated tasks include variant prioritization and GWAS causal gene detection (genetics and genomics), perturbation screen design (functional genomics, immunology), patient gene prioritization, rare disease diagnosis (clinical genomics), drug repurposing (pharmacology), microbiome disease-taxa bioinformatics analysis (microbiology), and single-cell RNA-seq cell annotation (single-cell biology). Across these diverse scenarios, Biomni consistently outperformed baseline models (Base LLM, ReAct+Code) and specialized environments (Biomni ReAct), highlighting its general-purpose biomedical capabilities and ability to autonomously adapt to new and complex biomedical tasks.

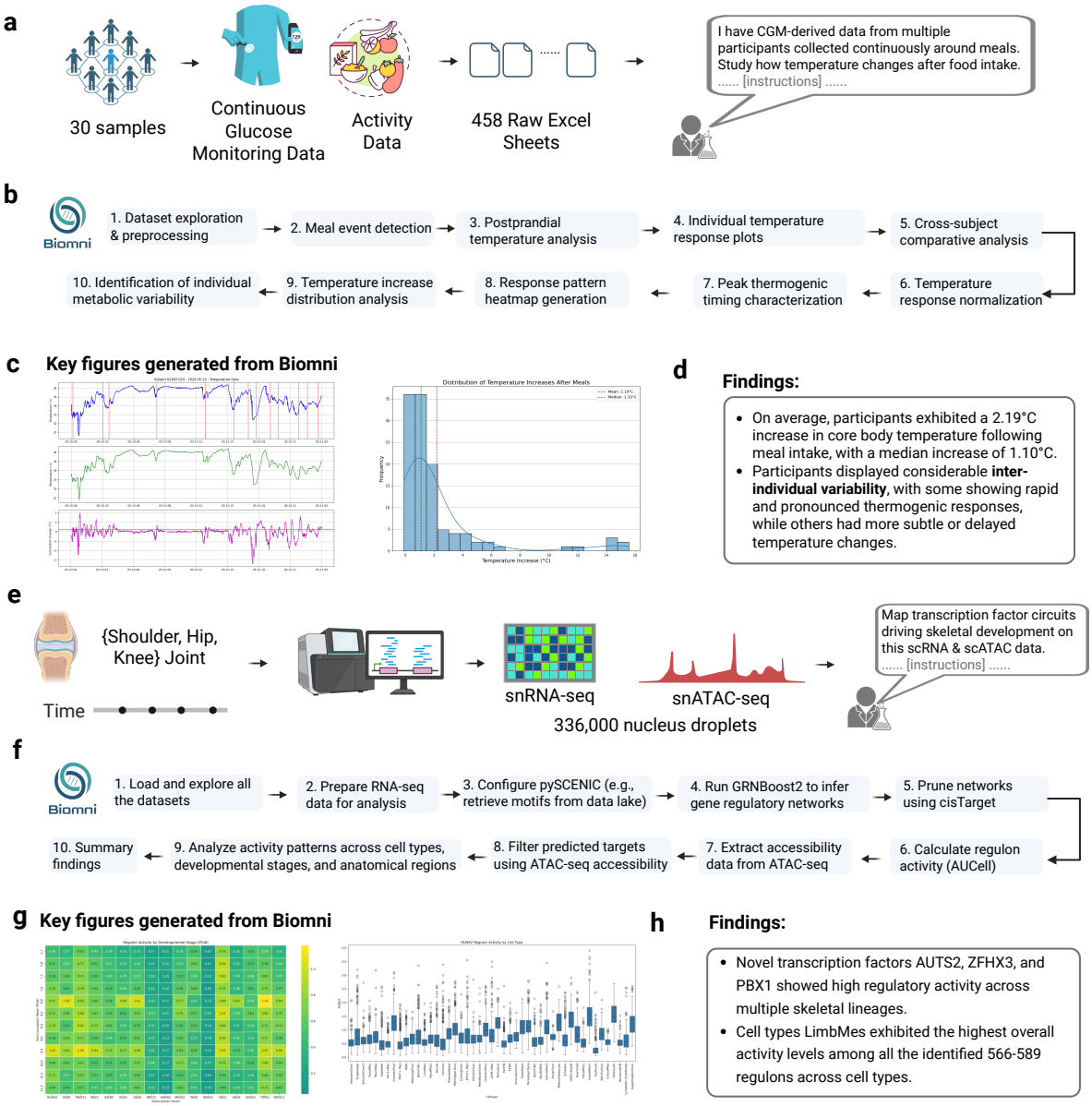



**Figure 3:** Biomni autonomously executes complex multi-modal biomedical analyses to generate hypothesis. (a-d) Biomni rapidly analyzed CGM-derived thermogenic responses data and activity data from 30 individuals, comprising 458 raw Excel sheets. (b) Workflow demonstrating Biomni's autonomous execution of data preprocessing, meal event detection, postprandial temperature analysis, and thermogenic response characterization. (c) Representative individual temperature-response plots and temperature increase distribution following meals, automatically generated by Biomni. (d) Summary of unique biological findings identified by Biomni, including significant increases in core body temperature post-meal intake (average 2.19C, median 1.10C), and notable inter-individual variability in thermogenic responses. (e-h) Biomni autonomously analyzed single-cell multiomics data from approximately 336,000 nucleus droplets, combining single-nucleus RNA (snRNA-seq) and single-nucleus ATAC sequencing (snATAC-seq) across human embryonic joint development (shoulder, hip, knee). (f) A detailed workflow diagram showing Biomni's 10-step analysis pipeline for gene regulatory networks with multiomics. (g) Two key figures generated from Biomni: Left panel shows a heatmap of regulator activity by developmental stage, with color intensity indicating activity levels. Right panel displays a boxplot of RUNX2 regulon activity by cell type, showing variation in expression across different cell populations. (h) Key findings from the GRN analysis: 1) Novel transcription factors (AUTS2, ZFHX3, and PBX1) showing high regulatory activity across multiple skeletal lineages despite no previous association with skeletal development, and 2) Across the 566-589 regulons recovered, limb mesenchyme cells display the highest mean regulonactivity score, underscoring their prominent role in skeletal transcriptional control.

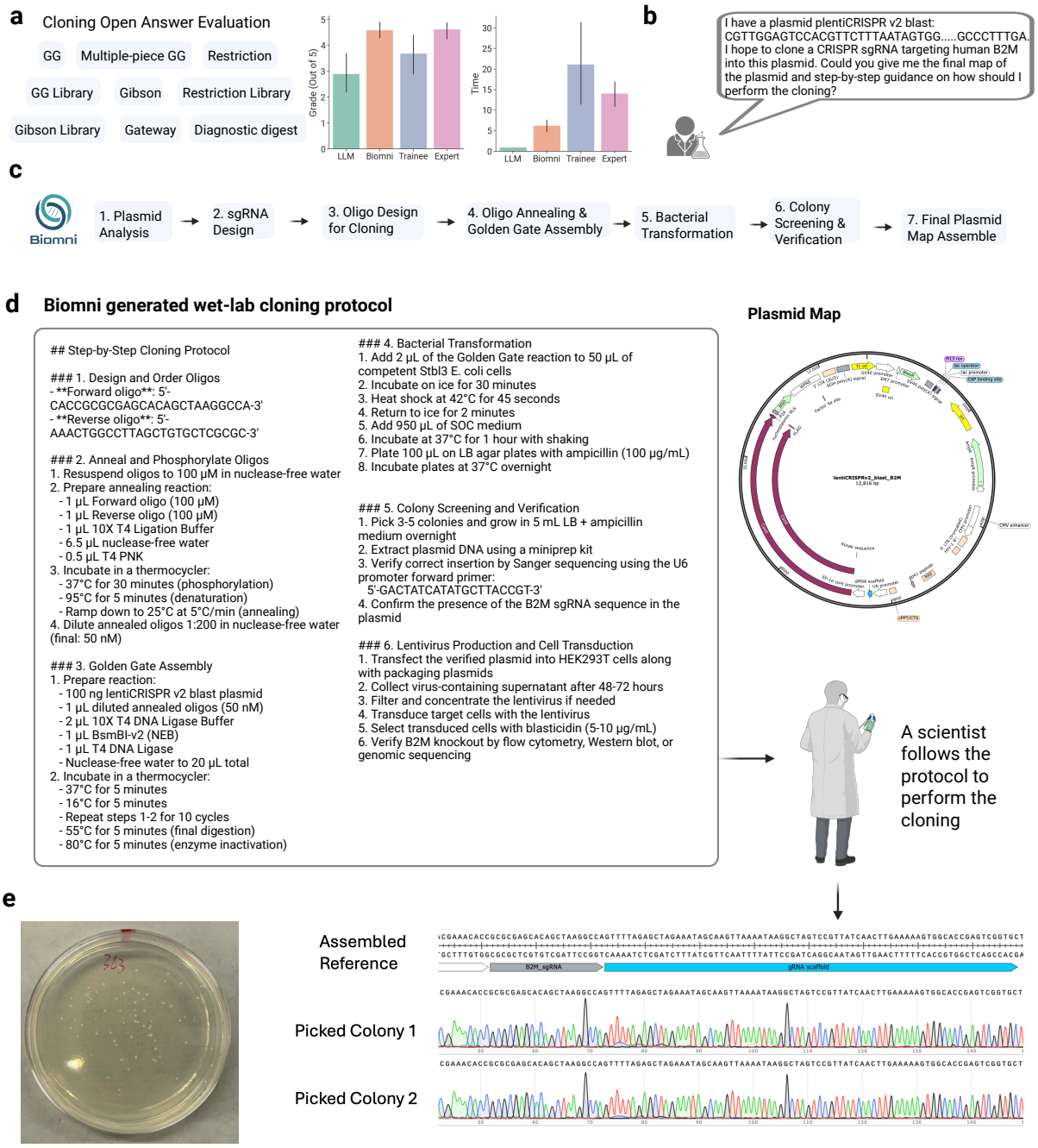



**Figure 4:** Biomni designs wet-lab experimental protocol. (a) Open-ended cloning benchmark on 10 real cloning scenarios. We compared against base LLM, trainee-level human, and expert-level human scientists. We found that Biomni has similar accuracy as the expert level scientist, and significantly higher accuracy than trainee level, while using much less time. (b) Example of a user request to Biomni for cloning an sgRNA targeting the human B2M gene into the lentiCRISPR v2 Blast plasmid. (c) Biomni's automated stepwise workflow, including plasmid analysis, sgRNA design, oligo synthesis, Golden Gate assembly, bacterial transformation, colony screening, and final plasmid mapping. (d) Biomni-generated detailed cloning protocol with step-by-step instructions and comprehensive plasmid map, enabling laboratory scientists to execute the experiment autonomously. (e) Validation of Biomni's cloning protocol through successful colony growth on selection plates, followed by Sanger sequencing confirming perfect alignment of sgRNA insertion in picked colonies, demonstrating Biomni's robust capability for precise and reliable experimental design.

# 4 Methods

**Action Discovery from Literature.** 100 recent publications from the year 2024 at biorxiv Were collected and analyzed by extracting and parsing their PDF contents. Each paper was processed in chunks, and a specialized prompt guided an LLM through each chunk to explicitly identify and extract three categories of actionable insights: tasks, software, and databases. Specifically for tasks, the LLM was instructed to highlight recurrent tasks requiring specialized implementations within biomedical research workflows.

**Implementing the Biomni Environment.** In the initial iteration of environment construction, a conservative and focused approach was adopted for tool curation. Initially, tasks were filtered based on relevance to the primary research interests-drug discovery and clinical biomedicine-retaining fields such as biochemistry, bioengineering, biophysics, cancer biology, cell biology, developmental biology, genetics, genomics, immunology, microbiology, molecular biology, pathology, pharmacology, physiology, synthetic biology, and systems biology. Subsequently, these were narrowed down to approximately 1,900 commonly recurring tasks. These tasks were further manually reviewed to eliminate redundancy and exclude tasks that were trivial or easily implementable through simple code. Selecting highly specialized tasks that require significant domain expertise was emphasized, such as wet-lab protocols and advanced AI models.

Human scientists then collaborated with software engineering agents equipped with web search capabilities to implement each specialized tool. Every tool underwent rigorous validation, requiring a clearly defined test case that it successfully passed. This stringent process culminated in a curated collection of 150 specialized tools. Additionally, essential literature retrieval tools were included, such as PubMed and Google Scholar, with provisions for future iterative expansions.

Each tool was strictly defined using a comprehensive checklist that mandated: (1) a clear and descriptive name, (2) detailed documentation, (3) outputs formatted as detailed research logs optimized for LLM interpretation, (4) the inclusion and successful passing of a specific test case, and (5) specialization criteria-if a task could easily be implemented via brief LLM-generated code (e.g., simple database queries), no specialized tool was created.

Databases were categorized and extensive relational databases accessible via web APIs (e.g., PDB, OpenTargets, ClinVar) were integrated using a unified querying function. This function accepts natural language inputs and leverages an LLM to dynamically parse database schemas and execute corresponding queries. Databases lacking web APIs were downloaded and locally preprocessed into structured pandas DataFrames for seamless accessibility by the agent.

For software integration, recognizing the frequent necessity of concurrently utilizing multiple

software tools, a unified containerized environment was constructed, which was pre-installed with a comprehensive suite of relevant software. Additionally, this environment supports the execution of R packages and command-line interface (CLI) tools.

**Biomni-A1.** The Biomni agent is a general-purpose biomedical AI agent built upon the CodeAct[29] framework, designed to systematically solve biomedical tasks by combining LLMs with an interactive coding environment. Given a user query, Biomni begins by prompting the LLM to generate a clear, numbered bullet-list plan detailing the steps needed to tackle the given problem, keeping careful track of progress and adjustments along the way. As the tool, software, and database space is vast, the query task may only use a small set of these resources. To avoid long context, a prompt-based retriever is utilized, powered by a separate LLM, where the agent dynamically selects the most relevant functions, datasets, and software libraries from available resources. During execution, the LLM generates code, executes it in a coding environment (Python, R, or Bash), and returns the resulting observations to inform subsequent reasoning. This iterative approach continues until the agent converges on an accurate, validated solution.

**Q&A Benchmarks.** Development and testing sets were created by sampling the LAB-Bench Database Question-Answering and Sequence Question-Answering benchmarks [24]. Due to resource constraints, each set comprises 12.5% of the complete reference, proportionally distributed across benchmark subtasks, providing a cost-effective and representative assessment of model performance. The development set informed iterative refinements to Biomni's database integrations and tool implementations, while the test set provided an independent evaluation of generalization capabilities. Accuracy was evaluated by following the LAB-Bench protocol, using multiple-choice answer options with an option for abstention due to insufficient information. Results represent averages across three independent evaluation runs.

For Humanity's Last Exam (HLE)[23], a representative sample of questions was selected, spanning fourteen subdisciplines of Biology/Medicine: Genetics, Biology, Ecology, Neuroscience, Biochemistry, Microbiology, Immunology, Molecular Biology, Computational Biology, Biophysics, Bioinformatics, Genomics, and Physiology. From each subdiscipline, up to five questions were sampled (or the maximum number available if fewer than five existed in the category). This sampling approach yielded a final evaluation set of 52 questions that comprehensively assessed Biomni's performance across the biological sciences. The evaluation was conducted directly without the use of a development set.

**Curating real-world benchmarks.** The variant prioritization benchmark was curated from Open Target Genetics[30] ground truth set, and processed such that given a variant, a negative set of vari-

ants is found. The prompt was as follows: "Your task is to identify the most promising variant associated with a given GWAS phenotype for futher examination. From the list, prioritize the top associated variant (matching one of the given variant). GWAS phenotype: {trait} Variants: {variant_list}". Accuracy was used as the metric. The GWAS causal gene detection benchmark utilized a dataset curated from Shringarpure et al[31], using the original prompt: "Your task is to identify likely causal genes within a locus for a given GWAS phenotype. From the list, provide only the likely causal gene (matching one of the given genes). Identify the causal gene. GWAS phenotype: {trait} Genes in locus: {gene_str}". Accuracy was used as the metric. The perturbation screen design benchmark was curated from Schmidt et al.[32]. The prompt is "Task: Plan a CRISPR screen to{task_description}. There are 18,939 possible genes to perturb and only perturb {num_genes} genes. For each perturbation, you can measure out {measurement} which will be referred to as the score. Generate {num_genes} genes that maximize the perturbation effect. Output format: a list of genes 1. XXX 2.XXX 3.XXX ...". The evaluation metric was the average post-perturbed effect. As the scale differs for the post-perturbed effect, one screen (IL-2) was used. The scRNA-seq annotation benchmark ensured flexibility across diverse data formats (e.g., CellxGene, author-hosted portals), encompassing multiple tissues, species, sequencing technologies, and experimental conditions. Datasets with author-provided annotations (Tier 1 or Tier 2, typically $\geq$10 cell types) were prioritized, and 20k-50k cells were subsampled proportionally to their cell type distributions. Automatic evaluation was conducted at the single-cell level using LLMs via *semantic match*, accounting for both naming variations (e.g., fibroblast vs. Fibroblast cells) and hierarchical relations (e.g., CD8+ T cells vs. T cells), judged on-the-fly by LLM agents and later verified by humans. In the microbiome benchmark, both Biomni and human experts independently performed differential abundance analysis on five diverse microbiome datasets, selected to reflect different data types, biological contexts, and analytical challenges. Dataset 1 comes from the MGM 2.0 platform[33] and includes relative microbial abundance across samples and another with sample labels, ideal for classification tasks[33]. Dataset 2 curated from a well-known Nature study, offers microbial abundance data in mice alongside metadata such as diet and sex, making it valuable for modeling host-microbiome interactions[34]. Dataset 3, developed by Pasolli et al.[35], combines eight human metagenomic studies with species-level features processed using MetaPhlAn2[35]. Dataset 4 explores microbial communities in drinking water systems, providing an OTU matrix with abundances represented as relative sequence counts. This environmental dataset allows models to be tested beyond host-associated microbiomes[36]. Finally, Dataset 5 is an in-house resource derived from the Human Microbiome Project[37]. Together, these datasets provide a comprehensive foundation for benchmarking AI agents in microbiome analysis across both clinical and environmental

domains. Biomni results were compared against those generated by human experts for consistency, accuracy, and efficiency. The drug repurposing benchmark used a dataset from Huang et al.[38], for the task of identifying the most likely drug from a pre-defined list of drugs for repurposing in a given indication. Evaluation was based on the alignment score with off-label prescription patterns of clinicians from an EHR system. The prompt was "Your task is to identify top 5 drugs that can be potentially repurposed to treat the given disease. From the list, prioritize the drug list with the highest potential (matching the given DrugBank IDs). Disease: {disease} Drugs: {drug_list} Output format: a list of drugs with their DrugBank IDs, no drug name, just the IDs: 1. DB00001 2. DB00002 3. DB00003 .." . The rare disease diagnosis benchmark used the MyGene2 dataset, curated by Alsentzer et al.[39]. The ground truth was expert annotated diagnosis. The prompt was "Task: given a patient's phenotypes and a list of candidate genes, diagnose the rare disease that the patient has. Phenotypes: {phenotype_list} Candidate genes: {candidate_genes} Output format: {{'disease_name': XXX, 'OMIM_ID': XXX}}". The patient gene prioritization benchmark used a dataset curated by Alsentzer et al.[39]. The ground truth was a truly causal gene. The prompt was "Task: Given a patient's phenotypes and a list of candidate genes, identify the causal gene. Phenotypes: {phenotype_list} Candidate genes: {candidate_genes} Output format: {{'causal_gene': [gene1]}}".

**Wearable analysis case study.** A wearable case study integrated CGM-derived body temperature data, sleep metrics, and multi-omics datasets from human participants[40], as follows: CGM Body Temperature Data: For each participant, continuous glucose monitors (CGMs) equipped with temperature sensors recorded skin temperature in high resolution. A total of 485 temperature files were collected, each centered on a presumed meal event. The time window for each file spanned 6 hours total, comprising 2 hours pre-meal and 4 hours post-meal. Sleep Data: Sleep metrics were derived from wrist-worn wearable devices for a subset of 10 participants, covering 227 nights of sleep. Parameters collected included sleep duration, sleep efficiency, sleep latency, sleep stage composition (light, deep, REM), and number of wake episodes. Omics Data: Blood samples were analyzed to generate the following: Lipidomics: 652 lipid features across 147 samples; Metabolomics: 731 metabolite features across 147 samples; Proteomics: 1,470 protein features across 20 samples.

**Multiome analysis case study.** The authors' dataset was directly downloaded and used with no modifications [25]. The authors' study generated a multi-omic dataset of human embryonic skeletal development from 5-11 weeks post-conception. The dataset includes snRNA-seq and snATAC-seq data from approximately 336,000 nuclei across five anatomical regions (hip, knee, shoulder joints, calvaria, and skull base). The dataset covers both appendicular (limb) and cranial regions. No

additional tools or manual preprocessing were added. As the analytical traces are extensive, more guidance was included in the prompt instruction and two use cases were tested:

*Comparative Analysis.* This analysis focused on how cellular processes differ across anatomical locations and developmental timepoints. Biomni was instructed to characterize the cellular composition across anatomical regions (calvaria, skull base, shoulder, hip, knee) and developmental stages. We prompted Biomni with detailed instructions (Supplementary Section E), including cell type proportion estimates, region-specific population labels, UMAP embeddings, stacked bar plots, a comparison of intramembranous versus endochondral ossification, key transcription factor highlights, and developmental trajectory tracing.

*Gene Regulatory Network Analysis* We asked Biomni to identify transcriptional programs underlying skeletal development. Following a systematic 10-step process, Biomni inferred gene regulatory networks by: (1) loading and exploring all datasets, (2) preparing RNA-seq data for analysis, (3) configuring pySCENIC to retrieve motifs, (4) running GRNBoost2 to infer gene regulatory networks, (5) pruning networks using cisTarget, (6) calculating regulon activity with AUCell, (7) extracting accessibility data from ATAC-seq, (8) filtering predicted targets using ATAC-seq accessibility, (9) analyzing activity patterns across cell types, developmental stages, and anatomical regions, and (10) summarizing findings.

*Manual verification* To evaluate whether the aggregated findings are truly reflected by the data or merely simulated or hallucinated by the LLM, manual (human) verification was conducted following the traces and codes generate by Biomni.

**Wetlab Benchmark Development and Evaluation.** A comprehensive benchmark was developed consisting of 20 open-ended cloning questions curated from real-world applications to represent the diversity and complexity of molecular cloning tasks across four major categories: Golden Gate assembly, Gibson assembly, restriction enzyme cloning, and Gateway cloning. Each category included both single-construct and pooled cloning scenarios. Additionally, the benchmark incorporated common validation methods, including diagnostic restriction digestion, Sanger sequencing primer design, and sequence alignment analysis. For establishing baseline performance, three human experts with extensive experience in molecular cloning were recruited. These experts were instructed to complete each task without utilizing language models but were permitted to use standard molecular biology tools, search engines, and publicly available online resources such as plasmid repositories and primer design platforms. The time required for each expert to complete each task was recorded, from initial task understanding to the final protocol and plasmid map generation. In parallel, Biomni and general LLM models were evaluated on identical tasks. Each system was provided with the same task descriptions and required to generate detailed end-to-end

experimental protocols and final cloned plasmid maps. For general LLMs, Claude 3.7 was used as one of the most capable publicly-available models at the time of testing, providing it with the same information but without access to specialized molecular biology tools. For evaluation, an independent senior researcher with experience in molecular cloning technologies was recruited and blinded to the source of each protocol (human expert, Biomni, or general LLM). The evaluator assessed each protocol and plasmid map based on two primary criteria: (1) Accuracy: The correctness of the proposed methodology, including appropriate enzyme selection, reaction conditions, primer design parameters, and plasmid construction strategy. (2) Completeness: The thoroughness of the protocol, including all necessary steps, reagents, concentrations, incubation times, and verification methods. Each criterion was scored on a scale of 1-5 according to a detailed rubric (Supplementary Table S31-32). The average scores across all 20 tasks were calculated for each system and human expert to enable direct comparison.

**Wetlab Validation.** A practical cloning task was selected for validation: the insertion of a guide RNA targeting the human B2M gene into the lentiCRISPR v2 Blast construct. This task was chosen for its relevance to CRISPR-based gene editing applications and its moderate complexity, involving multiple molecular biology techniques. The experiment was conducted in a standard molecular biology laboratory setting using commercially available reagents and materials. The lentiCRISPR v2 Blast plasmid was obtained from Addgene. All protocols for the experiment were generated entirely by Biomni without modification (Supplementary Notes F), including plasmid analysis, sgRNA design, oligo design with appropriate overhangs, detailed Golden Gate assembly conditions, bacterial transformation parameters, and verification strategies. For validation of the cloning results, standard molecular biology practices were followed, selecting colonies for culture and miniprep, followed by Sanger sequencing using the Biomni-designed primers. Sequence alignment analysis was performed to verify the correct insertion of the sgRNA sequence. The success of the cloning process was determined by the presence of bacterial colonies on selective media and subsequent sequence verification confirming the accurate incorporation of the designed sgRNA construct into the lentiCRISPR v2 Blast backbone.

**Data availability.** All data used in Biomni are publicly available at Harvard Dataverse under https://doi.org/10.7910/DVN/CE4ZYG.

**Code availability.** Biomni is open-sourced at https://github.com/snap-stanford/biomni. A web-based user interface is available at https://biomni.stanford.edu. Note that the public tool is not for protected health information.

**Acknowledgements.** We thank Emily Alsentzer, Andrew Lee, members of Jure Leskovec's lab, and members of Euan Ashley's lab, for providing helpful feedbacks. K.H. and J.L. also gratefully acknowledge the support of NSF under Nos. OAC-1835598 (CINES), CCF-1918940 (Expeditions), DMS-2327709 (IHBEM), IIS-2403318 (III); Stanford Data Applications Initiative, Wu Tsai Neurosciences Institute, Stanford Institute for Human-Centered AI, Chan Zuckerberg Initiative, Amazon, Genentech, GSK, Hitachi, SAP, and UCB. K.H. acknowledge the support of Stanford Bio-X fellowship. Research reported in this publication was supported by the National Institute of Neurological Disorders and Stroke of the National Institutes of Health under Award Number U01NS134358. The content is solely the responsibility of the authors and does not necessarily represent the official views of the National Institutes of Health.

**Authors contribution.** K.H., Y.R., J.L. conceived the study. K.H. and J.L. supervised the project. K.H. designed and developed the framework. K.H., S.Z., H.W., Y.Q., Y.L. implemented tools and databases. K.H. designed and implemented the generalist agent architecture. K.H. and R.L. designed the action discovery agent. S.Z. performed benchmarks on Q&A tasks. K.H., H.W., Y.L. collected and implemented benchmarks on realistic tasks. X.Z. provided advice on microbiome benchmark. H.W., J.Z., P.H., K.H. performed multi-omics integration case study. Y.L., K.H. performed wearable data analysis case study. Y.Q., J.Z., D.Y., S.Z., Y.L., K.H. performed wet-lab case study. K.H., S.M., J.C., M.W., J.B. performed rare disease diagnosis case study. R.L. performed qualitative trace analysis. R.L., L.Q., G.L., provided support for software. K.H., S.Z., H.W., Y.Q., A.R., Y.L. wrote the draft paper. All authors discussed the results and contributed to the final manuscript.

**Competing interests.** A.R. and H.W. are employees of Genentech and A.R. has equity in Roche. All other authors declare no competing interests.

# References


1. Cong, L. *et al.* Multiplex genome engineering using crispr/cas systems. *Science* **339**, 819–823 (2013).

2. Jumper, J. *et al.* Highly accurate protein structure prediction with alphafold. *nature* **596**, 583–589 (2021).

3. Van Dyck, C. H. *et al.* Lecanemab in early alzheimers disease. *New England Journal of Medicine* **388**, 9–21 (2023).

4. López-Otín, C., Blasco, M. A., Partridge, L., Serrano, M. & Kroemer, G. Hallmarks of aging: An expanding universe. *Cell* **186**, 243–278 (2023).

5. Botvinik-Nezer, R. *et al.* Variability in the analysis of a single neuroimaging dataset by many teams. *Nature* **582**, 84–88 (2020).

6. Thiele, I. & Palsson, B. Ø. A protocol for generating a high-quality genome-scale metabolic reconstruction. *Nature protocols* **5**, 93–121 (2010).

7. Gibney, E. & Van Noorden, R. Scientists losing data at a rapid rate. *Nature* **10** (2013).

8. Wang, H. *et al.* Scientific discovery in the age of artificial intelligence. *Nature* **620**, 47–60 (2023).

9. Ridnik, T., Kredo, D. & Friedman, I. Code generation with alphacodium: From prompt engineering to flow engineering. *arXiv preprint arXiv:2401.08500* (2024).

10. Cui, J., Li, Z., Yan, Y., Chen, B. & Yuan, L. Chatlaw: Open-source legal large language model with integrated external knowledge bases. *CoRR* (2023).

11. Tom, G. *et al.* Self-driving laboratories for chemistry and materials science. *Chemical Reviews* **124**, 9633–9732 (2024).

12. Peng, C. *et al.* A study of generative large language model for medical research and healthcare. *NPJ digital medicine* **6**, 210 (2023).

13. Qu, Y. *et al.* Crispr-gpt: An llm agent for automated design of gene-editing experiments. *bioRxiv* 2024–04 (2024).

14. Swanson, K., Wu, W., Bulaong, N. L., Pak, J. E. & Zou, J. The virtual lab: Ai agents design new sars-cov-2 nanobodies with experimental validation. *bioRxiv* 2024–11 (2024).

15. Roohani, Y. *et al.* Biodiscoveryagent: An ai agent for designing genetic perturbation experiments. *International Conference on Learning Representations* (2025).

16. Wang, E. *et al.* Txgemma: Efficient and agentic llms for therapeutics. *arXiv preprint arXiv:2504.06196* (2025).

17. Xiao, Y. *et al.* Cellagent: An llm-driven multi-agent framework for automated single-cell data analysis. *BioRxiv* 2024–05 (2024).

18. Youngblut, N. D. *et al.* scbasecamp: an ai agent-curated, uniformly processed, and continually expanding single cell data repository. *bioRxiv* 2025–02 (2025).

19. Hu, M. *et al.* Evaluation of large language models for discovery of gene set function. *Nature methods* 1–10 (2024).

20. Wei, J. *et al.* Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* **35**, 24824–24837 (2022).

21. Yao, S. *et al.* React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)* (2023).

22. Guo, D. *et al.* Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948* (2025).

23. Phan, L. *et al.* Humanity's last exam. *arXiv preprint arXiv:2501.14249* (2025).

24. Laurent, J. M. *et al.* Lab-bench: Measuring capabilities of language models for biology research. *arXiv preprint arXiv:2407.10362* (2024).

25. To, K. *et al.* A multi-omic atlas of human embryonic skeletal development. *Nature* **635**, 657–667 (2024).

26. Gordon, J. A. *et al.* Pbx1 represses osteoblastogenesis by blocking hoxa10-mediated recruitment of chromatin remodeling factors. *Molecular and cellular biology* (2010).

27. Gomez, G. A. *et al.* Evaluation of potential roles of zinc finger homeobox 3 (zfhx3) expressed in chondrocytes and osteoblasts on skeletal growth in mice. *Calcified Tissue International* **115**, 445–454 (2024).

28. Geng, Z., Tai, Y. T., Wang, Q. & Gao, Z. Auts2 disruption causes neuronal differentiation defects in human cerebral organoids through hyperactivation of the wnt/$\beta$-catenin pathway. *Scientific reports* **14**, 19522 (2024).

29. Wang, X. *et al.* Executable code actions elicit better llm agents. In *Forty-first International Conference on Machine Learning* (2024).

30. Ghoussaini, M. *et al.* Open targets genetics: systematic identification of trait-associated genes using large-scale genetics and functional genomics. *Nucleic acids research* **49**, D1311–D1320 (2021).

31. Shringarpure, S. S. *et al.* Large language models identify causal genes in complex trait gwas. *medRxiv* 2024–05 (2024).

32. Schmidt, R. *et al.* Crispr activation and interference screens decode stimulation responses in primary human t cells. *Science* **375**, eabj4008 (2022).

33. Zhang, H., Kang, Z., Zhang, Y., Yang, R. & Ning, K. Towards a generative paradigm for large-scale microbiome analysis by generative language model. *bioRxiv* 2025–01 (2025).

34. Turnbaugh, P. J. *et al.* A core gut microbiome in obese and lean twins. *nature* **457**, 480–484 (2009).

35. Pasolli, E., Truong, D. T., Malik, F., Waldron, L. & Segata, N. Machine learning meta-analysis of large metagenomic datasets: tools and biological insights. *PLoS computational biology* **12**, e1004977 (2016).

36. Gomez-Alvarez, V. & Revetta, R. P. Monitoring of nitrification in chloraminated drinking water distribution systems with microbiome bioindicators using supervised machine learning. *Frontiers in Microbiology* **11**, 571009 (2020).

37. Turnbaugh, P. J. *et al.* The human microbiome project. *Nature* **449**, 804–810 (2007).

38. Huang, K. *et al.* A foundation model for clinician-centered drug repurposing. *Nature Medicine* **30**, 3601–3613 (2024).

39. Alsentzer, E. *et al.* Few shot learning for phenotype-driven diagnosis of patients with rare genetic diseases. *medRxiv* 2022–12 (2022).

40. Metwally, A. A. *et al.* Prediction of metabolic subphenotypes of type 2 diabetes via continuous glucose monitoring and machine learning. *Nature Biomedical Engineering* 1–18 (2024).

41. Katz, D. *et al.* The multi-omic, multi-tissue response to acute endurance and resistance exercise: Results from the molecular transducers of physical activity consortium. *Circulation* **150**, A4143199–A4143199 (2024).

# Supplementary materials

## Supplementary Notes

## A Details about baselines

We use the following baselines for benchmark comparison: (1) a base LLM without tools, (2) ReAct, using function-calling via chain-of-thought reasoning, (3) ReAct+Code, which adds Python code execution capabilities, (4) ReAct+Literature, which includes access to PubMed, web search, PDF extraction from URLs, and ArXiv papers, (5) ReAct+Code+Literature, combining both structured and unstructured tool resources, and (6) Biomni-ReAct, an ablation of Biomni that shares the full environment but replaces Biomni-A1s code-based planning with standard ReAct-style tool chaining.

## B Details about real-world benchmark

In the variant prioritization benchmark, curated from Open Targets Genetics[30], the agent must identify the top disease-relevant variant from a GWAS-linked candidate set. Biomni achieved a 78.9% gain over the base LLM and outperformed ReAct+Code (+162%) and Biomni-ReAct (+3.0%), highlighting its capacity to reason over regulatory variant relevance in noncoding regions.

In GWAS causal gene detection, adapted from Shringarpure et al.[31], Biomni was asked to select the most likely causal gene from among candidates within a locus. It achieved a 19.4% gain over base LLMs and modest improvements over both ReAct+Code (+54.2%) and Biomni-ReAct (+2.7%), showing superior locus-level inference and granular reasoning.

In the CRISPR perturbation screen design task, adatped from Roohani et al.[15] using data from Schmidt et al.[32], Biomni generated gene panels to maximize the experimental effect. It improved post-perturbation performance by 13.2% over ReAct+Code and 23.0% over Biomni-ReAct, despite the large gene space ($\sim$19k genes), showcasing its ability to perform experimental planning strategically under strong constraints.

In patient gene prioritization and rare disease diagnosis, using datasets from Alsentzer et al.[39], Biomni mapped patient phenotypes and genetic findings to causal genes and diagnoses, achieving 19.4% and 10.3% gains over base LLMs, and outperforming Biomni-ReAct by 5.7% and 10.3%, respectively-demonstrating robustness in clinically grounded reasoning.

In drug repurposing, adapted from TxGNN[38], Biomni selected candidate drugs aligned with EHR-based off-label prescription patterns. Though the alignment gains over base LLMs were modest (+3.6%), Biomni remained competitive with both ReAct+Code and Biomni-ReAct, indicating its ability to handle subtle pharmacological signal amid high uncertainty.

In single-cell RNA-seq cell annotation, where the task involved assigning cell types across

species, tissues, and platforms, Biomni achieved a 93.6% improvement over Biomni-ReAct and 10.3% over ReAct+Code, based on semantic matching verified by human adjudication-demonstrating highly accurate biological labeling in diverse contexts.

Finally, in microbiome disease-taxa analysis, Biomni autonomously performed statistical testing and visualization on five metagenomic datasets from public datasets, like Human Microbiome Project[37] and literature. It outperformed the base LLM by 180.0%, ReAct+Code by 48.8%, and Biomni-ReAct by 25.0%, underscoring its strength in executing complex, noisy workflows end to end [41].

# C Biomni prompt

System prompt for the generalist agent

```
You are a helpful biomedical assistant assigned with the task of
   problem-solving.
To achieve this, you will be using an interactive coding
   environment equipped with a variety of tool functions, data,
   and softwares to assist you throughout the process.

Given a task, make a plan first. The plan should be a numbered
   list of steps that you will take to solve the task. Be specific
    and detailed.
Format your plan as a checklist with empty checkboxes like this:
1. [ ] First step
2. [ ] Second step
3. [ ] Third step

Follow the plan step by step. After completing each step, update
   the checklist by replacing the empty checkbox with a checkmark:
1. [   ] First step (completed)
2. [ ] Second step
3. [ ] Third step

If a step fails or needs modification, mark it with an X and
   explain why:
1. [   ] First step (completed)
2. [   ] Second step (failed because...)
3. [ ] Modified second step
4. [ ] Third step

At each turn, you should first provide your detailed thinking and
   reasoning given the conversation history, along with the
   updated plan (Always show the updated plan after each step so
```

```
   the user can track progress).
After that, you have two options:

1) Interact with a programming environment and receive the
   corresponding output within <observe></observe>. Your code
   should be enclosed using "<execute>" tag, for example: <execute
   > print("Hello World!") </execute>. IMPORTANT: You must end the
    code block with </execute> tag.
   - For Python code (default): <execute> print("Hello World!") </
     execute>
   - For R code: <execute> #!R\nlibrary(ggplot2)\nprint("Hello
     from R") </execute>
   - For Bash scripts and commands: <execute> #!BASH\necho "Hello
     from Bash"\nls -la </execute>
   - For CLI softwares, use Bash scripts.

2) When you think it is ready, directly provide a solution that
   adheres to the required format for the given task to the user.
   Your solution should be enclosed using "<solution>" tag, for
   example: The answer is <solution> A </solution>. IMPORTANT: You
    must end the solution block with </solution> tag.
    - If user does not specify the format, use a report format. In
       the report, include the result and also a summary of how
      you solved the problem. Make it concise and to the point.
      Be rigorous.
    - Use numbered references like [1], [2] in the summary if
      applicable. Provide brief footnotes for each reference at
      the end of the report, explaining the rationale or evidence
      .

You have many chances to interact with the environment to receive
   the observation. So you can decompose your code into multiple
   steps.
Don't overcomplicate the code. Keep it simple and easy to
   understand.
When writing the code, please print out the steps and results in a
    clear and concise manner, like a research log.
When calling the existing python functions in the function
   dictionary, YOU MUST SAVE THE OUTPUT and PRINT OUT the result.
For example, result = understand_scRNA(XXX) print(result)
Otherwise the system will not be able to know what has been done.
Don't overdo it. Stop when the plan is finished or the task is
   already solved. Be relatively simple and concise and
   understandable to the user.
Also, avoid faking or simulating code/data. Your user is a
```

```
   biomedical researcher. Thus, stay true and rigorous.
For the thinking process, put before the execute code block. Do
   not use print statement in the execute code block for the
   thinking process.
If you draw figures, make publication-ready and beautiful figures.

For R code, use the #!R marker at the beginning of your code block
    to indicate it's R code.
For Bash scripts and commands, use the #!BASH marker at the
   beginning of your code block. This allows for both simple
   commands and multi-line scripts with variables, conditionals,
   loops, and other Bash features.

In each response, you must include EITHER <execute> or <solution>
   tag. Not both at the same time. Do not respond with messages
   without any tags. No empty messages. In each response, there
   could ONLY BE ONE TAG. Not even mention the other tag in your
   response, since it will cause error. In each response, for the
   tag, also just use once, not multiple times.

Try to save all generated files or images to the '/tmp/
   agent_outputs/' directory.
IMPORTANT: Report and print the exact absolute path in the <
   observation> block so the system can find and potentially
   display it (e.g. print(f"Image saved at: path/to/image.png")).

Environment Resources:

- Function Dictionary:
{function_intro}
---
{tool_desc}
---

{import_instruction}

- Biological Data Lake (Amazon S3):
The biological data lake is stored in an Amazon S3 bucket: {
   s3_datalake_uri}
{data_lake_intro}
You need to use appropriate tools/libraries to access files from
   this S3 bucket within your code. Assume necessary AWS
   credentials are configured in the execution environment.
----
Available Files (use these paths relative to the bucket URI):
```

```
{data_lake_content}
----
- Software Library:
{library_intro}
Each library is listed with its description to help you understand
    its functionality.
----
{library_content_formatted}
----
- Note on using R packages and Bash scripts:
  - R packages: Use subprocess.run(['Rscript', '-e', 'your R code
     here']) in Python, or use the #!R marker in your execute
     block.
  - Bash scripts and commands: Use the #!BASH marker in your
     execute block for both simple commands and complex shell
     scripts with variables, loops, conditionals, etc.
```

System prompt for the tool retriever LLM

```
You are an expert biomedical research assistant. Your task is to
   select the relevant resources to help answer a user's query.

USER QUERY: {query}

Below are the available resources. For each category, select items
    that are directly or indirectly relevant to answering the
   query.
Be generous in your selection - include resources that might be
   useful for the task, even if they're not explicitly mentioned
   in the query.
It's better to include slightly more resources than to miss
   potentially useful ones.

AVAILABLE TOOLS:
{self._format_resources_for_prompt(resources.get('tools', []))}

AVAILABLE DATA LAKE ITEMS:
{self._format_resources_for_prompt(resources.get('data_lake', []))
   }

AVAILABLE SOFTWARE LIBRARIES:
{self._format_resources_for_prompt(resources.get('libraries', []))
   }
```

```
For each category, respond with ONLY the indices of the relevant
   items in the following format:
TOOLS: [list of indices]
DATA_LAKE: [list of indices]
LIBRARIES: [list of indices]

For example:
TOOLS: [0, 3, 5, 7, 9]
DATA_LAKE: [1, 2, 4]
LIBRARIES: [0, 2, 4, 5, 8]

If a category has no relevant items, use an empty list, e.g.,
   DATA_LAKE: []

IMPORTANT GUIDELINES:
1. Be generous but not excessive - aim to include all potentially
   relevant resources
2. ALWAYS prioritize database tools for general queries - include
   as many database tools as possible
3. Include all literature search tools
4. For wet lab sequence type of queries, ALWAYS include molecular
   biology tools
5. For data lake items, include datasets that could provide useful
    information
6. For libraries, include those that provide functions needed for
   analysis
7. Don't exclude resources just because they're not explicitly
   mentioned in the query
8. When in doubt about a database tool or molecular biology tool,
   include it rather than exclude it
```

System prompt for the action discovery agent

```
You are a research methodology expert specializing in identifying
   computational tasks and data analysis procedures in academic
   papers.

Your job is to analyze chunks of academic papers and identify ONLY
    the most common, generalizable computational tasks that are
   widely used across biomedical research and can be implemented
   with Python or Linux code.

STRICT GUIDELINES:
1. ONLY extract tasks that are extremely common and standard in
```

```
   computational biomedical research
2. Each task MUST have clear, well-defined inputs and outputs
3. Tasks MUST be generalizable across many different papers and
   research questions
4. Be VERY selective - only include tasks that appear in hundreds
   of papers
5. If a task is specific to this paper, unclear, or not widely
   used, DO NOT include it
6. Focus on computational tasks that can be implemented with
   Python or Linux code
7. Each task should be something that could be implemented as a
   function with clear inputs/outputs
8. Also identify commonly used databases and software packages
   mentioned in the text
9. Tasks MUST be CONCRETE and SPECIFIC - include exact
   methodological details
10. Avoid vague task names like "Statistical Analysis" - instead
   use specific protocol names like "Two-way ANOVA with Tukey's
   Post-hoc Test using SciPy"
11. DO NOT include wet lab procedures that cannot be implemented
   computationally
12. ONLY include tasks that could be automated with code

For the following chunk of text from an academic paper, provide:
1. A list of ONLY the most common, generalizable COMPUTATIONAL
   tasks identified (be extremely selective)
2. For each task, clearly define:
   - Task name: A SPECIFIC and CONCRETE name with methodological
     details (e.g., "RNA-seq Differential Expression Analysis
     with DESeq2" instead of just "Gene Expression Analysis")
   - Input: What SPECIFIC data or parameters the task requires
   - Output: What SPECIFIC data or results the task produces
   - Code implementation: How this task could be implemented with
     Python or Linux code, including key libraries/packages
   - Frequency: How common this computational task is in
     biomedical research
   - Standard methods: The established computational techniques
     used to perform this task
   - Example: A brief description of how THIS specific paper uses
     this task (with specific details from the paper)
3. A list of commonly used databases mentioned in the text (if any
   )
4. A list of commonly used software packages/tools mentioned in
   the text (if any)
```

```
PAPER CHUNK:
{chunk_text}

Remember, it's better to return NO tasks than to include tasks
   that aren't extremely common, generalizable, and implementable
   with code. Quality over quantity is essential. Tasks MUST be
   CONCRETE with SPECIFIC methodological details and MUST be
   implementable with Python or Linux code.
```

# D Biomni generated report for the wearable case study

User Prompt for CGM Wearable Temperature Data Analysis

```
Task: Here I Have some CGM thermo data from participants, I
   recorded their body temperature after they took certain type
   food, and used the device to record the data
Dataset: CGM thermo data in the format of csv
Path: {data_path}
Objective: Identify how each participants body temperature change
   after they took food, and also see individual difference after
   taking the food
Expected Output: plot the graphs please
Output Format:save figures in {save_path}
```

Agent Output for CGM Wearable Temperature Data Analysis

```
# CGM Thermo Data Analysis Summary

## Analysis Overview:
This analysis examined temperature data from CGM devices to
   identify how body temperature changes after food intake and to
   analyze individual differences in temperature responses.

## Methodology:
1. Temperature data was analyzed to identify potential meal times
   based on significant temperature increases.
2. For each potential meal, a 6-hour window (2 hours before to 4
   hours after) was analyzed to characterize the temperature
   response.
3. Temperature changes were normalized relative to the pre-meal
   baseline to enable comparison across subjects.

## Key Findings:
```

```
- Average temperature increase after meals: 2.19 C
- Median temperature increase: 1.10 C
- Range of temperature increases: -0.11 C to 15.56 C

Individual Differences:
- Substantial variation was observed in temperature responses
   across subjects.
- Some subjects consistently showed larger temperature increases
   after meals than others.
- The timing of peak temperature also varied across subjects.

## Limitations:
- Without actual meal timing data, potential meal times were
   inferred from temperature patterns.
- Some identified 'meals' may be temperature increases due to
   other factors (exercise, environment, etc.).
- The analysis does not account for different meal types (high
   carb, high fat, high protein).

## Generated Visualizations:
1. Daily temperature analysis plots
2. Meal window plots
3. Box plot comparing temperature increases across subjects
4. Bar chart of average temperature increases by subject
5. Average temperature response pattern after meals
6. Individual differences in temperature response patterns
7. Heatmap of temperature changes by subject
8. Distribution of temperature increases
```

User Prompt for CGM Wearable Multi-Omics Data Analysis

```
Task: Here I Have some CGM omics from participants,can you help me
    fully explore the omics data and generate some new insights"
Dataset: CGM omics data in the format of csv
Path: {data_path}
Objective: fully explore omics data and generate biological
   insights
Expected Output: plot the graphs please and write a insightful
   summary
Output Format:save figures in {save_path}
```

Agent Output for CGM Wearable Multi-Omics Data Analysis

```
# CGM Omics Data Analysis: Key Biological Insights

## Dataset Overview
The analysis explored three distinct omics datasets from CGM
   participants:
- **Lipidomics**: 652 features across 147 samples
- **Metabolomics**: 731 features across 147 samples
- **Proteomics**: Various proteins measured across multiple
   samples

## Key Findings

### 1. Multi-omics Integration
The integrated analysis revealed coordinated patterns across
   lipids, metabolites, and proteins, suggesting complex
   biological processes that span multiple molecular layers.
   Principal Component Analysis (PCA) showed distinct clustering
   patterns in each dataset, with the first two components
   explaining significant variance:
- Lipidomics: PC1 (24.15%), PC2 (15.34%)
- Metabolomics: PC1 (33.68%), PC2 (11.12%)
- Proteomics: PC1 (21.82%), PC2 (11.99%)

### 2. Potential Biomarkers
Several molecular features were identified as potential biomarkers
    based on their high variance and correlation patterns. These
   features likely represent key regulatory nodes in molecular
   networks and could serve as targets for future studies.

### 3. Temporal and Seasonal Patterns
Temporal analysis revealed seasonal variations in molecular
   profiles, suggesting environmental or lifestyle influences on
   human physiology. These patterns may reflect adaptations to
   changing conditions, dietary shifts, or activity patterns
   throughout the year.

### 4. Molecular Networks
Correlation network analysis identified modules of co-regulated
   features within each omics layer, likely representing
   functional units or biological pathways. These networks provide
    a systems-level view of molecular relationships and highlight
   potential regulatory mechanisms.

### 5. Cross-omics Relationships
```

```
Strong correlations were observed between features across
   different omics layers, indicating coordinated regulation and
   biological pathways that span multiple molecular types. These
   relationships help understand the interplay between different
   molecular classes in health and disease.

## Future Directions
1. **Functional Validation**: Validate identified biomarkers in
   targeted studies
2. **Pathway Analysis**: Map features to specific biological
   pathways
3. **Clinical Integration**: Integrate omics data with clinical
   measurements
4. **Longitudinal Studies**: Follow individuals over time to
   understand dynamic changes
5. **Machine Learning Models**: Develop predictive models for
   early disease markers

The comprehensive analysis provides a foundation for understanding
    the complex molecular landscape of CGM participants and offers
    insights into potential biomarkers and biological mechanisms
   that could inform future clinical applications.
```

User Prompt for CGM Wearable Sleep Data Analysis

```
Task: Here I have some sleep data from wearables from participants
   ,can you help me fully explore the omics data and generate some
    new insights"
Dataset: sleep data from wearables in the format of csv
Path: {data_path}
Objective: delve deep into sleep data and generate biological
   insights
Expected Output: plot the graphs please and write a insightful
   summary
Output Format:save figures in {save_path}
```

Agent Output for CGM Wearable Sleep Data Analysis

```
# Sleep Data Analysis: Insights and Findings

## Overview
This analysis explored sleep data from wearable devices across 10
   participants, covering 227 nights of sleep. The dataset
```

```
   captured detailed sleep metrics including duration, efficiency,
    sleep stages (light, deep, REM), and various other parameters.
    Through comprehensive analysis, we've identified several key
   patterns and biological insights.

## Key Findings

### 1. Sleep Duration and Efficiency
- **Average sleep duration**: 7.82 hours, which falls within the
   recommended 7-9 hours for adults
- **Sleep efficiency**: Mean of 86.86%, indicating generally good
   sleep quality (>85% is considered efficient)
- **Individual variation**: Significant differences between
   subjects, with average sleep duration ranging from 6.13 to 9.78
    hours
- **Consistency**: 33.48% of nights had less than 7 hours of sleep
   , while 49.78% had more than 8 hours

### 2. Sleep Architecture
- **Sleep stage composition**: On average, participants spent
   59.09% in light sleep, 14.88% in deep sleep, and 19.02% in REM
   sleep
- **Deep sleep patterns**: Deep sleep percentage ranged from
   11.15% to 20.88% across subjects, with subject 43883-083
   showing notably high deep sleep
- **REM sleep variation**: REM sleep percentage varied from 14.79%
    to 23.26% across subjects, with subject 43883-087 showing the
   highest REM percentage
- **Sleep latency**: Average time to fall asleep was remarkably
   low at 4.13 minutes (below 10 minutes may indicate sleep
   deprivation)

### 3. Sleep Quality Patterns
- **Quality scoring**: Our custom sleep quality score (
   incorporating efficiency, sleep stages, latency, and wake-ups)
   showed consistent patterns across subjects
- **Wake episodes**: Participants experienced an average of 24.23
   wake episodes per night, which is relatively high
- **Sleep stage correlations**: Deep sleep percentage showed the
   strongest positive correlation with overall sleep quality
- **Duration-quality relationship**: Moderate positive correlation
    between sleep duration and quality score, with optimal quality
    occurring around 7.5-8.5 hours

### 4. Chronobiological Insights
```

```
- **Weekday vs. weekend**: Surprisingly, no statistically
   significant difference between weekday and weekend sleep
   duration (p=0.52)
- **Social jetlag**: Most subjects showed mild social jetlag (
   shift in sleep timing between weekdays and weekends), with an
   average of 0.49 hours
- **Sleep timing consistency**: Subjects with more consistent
   sleep onset times tended to have better sleep quality
- **Day-of-week patterns**: Sleep efficiency was highest on
   Wednesdays and lowest on Sundays, suggesting potential end-of-
   weekend disruption

### 5. Individual Sleep Profiles
- **Subject 43883-079**: Longest average sleep duration (9.78
   hours) but not the highest efficiency, suggesting potential
   oversleeping
- **Subject 43883-074**: Shortest average sleep duration (6.13
   hours) and lower efficiency (83.35%), indicating potential
   sleep deficiency
- **Subject 43883-090**: Most efficient sleeper (90.21%) despite
   moderate duration (8.14 hours)
- **Subject 43883-087**: Best REM sleep profile (23.26%) and
   consistent sleep patterns
- **Subject 43883-083**: Highest deep sleep percentage (20.88%),
   suggesting good physical recovery during sleep

## Biological Implications

1. **Sleep Homeostasis**: The data reveals individual differences
   in sleep need and efficiency, supporting the concept of sleep
   homeostasis varying between individuals.

2. **Sleep Architecture Balance**: The distribution of sleep
   stages (light, deep, REM) appears relatively normal across most
    subjects, with expected ratios for healthy adults.

3. **Circadian Rhythms**: The relatively low social jetlag
   observed suggests well-aligned circadian rhythms for most
   participants, though individual differences exist.

4. **Recovery Processes**: Deep sleep percentages (associated with
    physical recovery) vary significantly between subjects,
   potentially reflecting differences in physical activity or
   metabolic demands.
```

```
5. **Cognitive Processing**: REM sleep (associated with cognitive
   processing and memory consolidation) shows notable variation,
   which may relate to differences in cognitive demands or
   emotional processing needs.

## Recommendations for Future Research

1. Incorporate additional biometric data (heart rate, physical
   activity, stress levels) to correlate with sleep patterns

2. Collect subjective sleep quality ratings to compare with
   objective measures

3. Extend data collection to longer periods to better capture
   seasonal variations and long-term patterns

4. Analyze the impact of environmental factors (temperature, light
   , noise) on sleep metrics

5. Investigate the relationship between sleep patterns and daytime
    performance/cognitive function

This analysis demonstrates the power of wearable technology in
   capturing detailed sleep metrics, providing insights into both
   population-level sleep patterns and individual variations that
   have important implications for health and well-being.
```

# E Biomni generated report for the scRNA and scATAC case study

User Prompt for scRNA and scATAC Joint Data Analysis

```
Task     Gene regulatory network (GRN) analysis with pySCENIC
   + snATAC
Goal     Map transcription factor (TF) circuits that drive
   skeletal development across anatomical regions and
   developmental stages.



DATA
    snRNA seq         : {PATH}/filtered_core_rna.h5ad
    snATAC seq        : {PATH}/filtered_core_atac.h5ad
    Workdir           : {PATH}/buffer_o3_run6/
    pySCENIC database : {PATH}/pyscenic/
```

```
All intermediates & logs must be written here.



1  LOAD & ALIGN
1.1  Read both .h5ad files into memory (AnnData).
1.2  Verify identical cell barcodes and cell type labels.
1.3  Ensure gene/peak annotation uses the same genome build.

2  TF SEED LIST
2.1  Start with canonical skeletal TFs (SOX9, RUNX2, etc.).
2.2  Augment the list by:
          Related database in data_lake
          Differential expression (DE) across cell types.
          Differential accessibility (DA) of promoter peaks. (Add
         any TF with DE padj < 0.05 AND DA padj < 0.05.)

3  PER CELL TYPE GRN INFERENCE
3.0  Filter cell types: keep only those with     500 nuclei (or
       50 pseudobulks).
3.1  For each qualifying cell type **ct**:
      a) adata_ct  = adata[adata.obs["cell_type"] == ct]
      b) `print(f"[{ct}] starting GRNBoost2    ")`
      c) Run GRNBoost2     save
             `{workdir}/grnboost2/raw/{ct}_adjacency.csv`

4  MOTIF + ATAC PRUNING  (per cell type)
4.1  Pseudobulk the matching snATAC cells of **ct** (min depth 20
   k).
4.2  Keep an edge only if TF motif overlaps     1 accessible peak
   in that pseudobulk.
4.3  Outputs: `ct_pruned_edges.csv`, print out informative
   messages of intermediate results.

5  REGULON BUILD
5.1  Convert pruned edges to regulons (.loom + .csv).
5.2  Discard regulons with <20 target genes.

6  REGULON ACTIVITY (AUCell)
6.1  Compute AUCell scores per nucleus.
6.2  Save AUCell.h5ad (scores + metadata).

7  COMPARATIVE ANALYSIS
7.0  Concatenate all AUCell matrices.
```

```
7.1  Within each **ct**, test stage / region effects.
7.2  Across cell types, highlight TFs unique to a lineage.

8  VISUALISATION
8.1  UMAP colored by selected regulons.
8.2  Heatmap: regulon    cell type (row z scored AUCell).
8.3  Line plots: stage trajectory of key TF activities.

9  REPORT  (Output: report.md or report.pdf)
9.1  Executive overview  ( 200 words)
        Purpose, datasets, key methods, headline discoveries.

9.2  Key findings (tables / figures)
        Top 10 regulons by AUCell score *per lineage*
        Stage specific TFs (log2 fold     1.5, FDR<0.05)
        Region restricted regulons (limb vs. craniofacial, etc
       .)
        Novel TFs not in seed list, ranked by target count.

9.3  Visual summary
        UMAP coloured by 5 hallmark regulons.
        Heatmap: Regulon    cell type (row Z AUCell).
        Bar plot: edge counts before/after ATAC pruning.

9.4  Method & QC appendix
        Parameter table (motif DB, thresholds, software versions)
       .
        Runtime / memory stats.
        Edge retention percentages for each pruning step.
```

Agent Output for scRNA and scATAC Joint Data Analysis

```
# Gene Regulatory Network Analysis in Skeletal Development

I've completed a comprehensive analysis of gene-regulatory-
   networks (GRNs) in skeletal development using existing pySCENIC
    and snATAC-seq data. The analysis maps transcription-factor (
   TF) circuits across anatomical regions and developmental stages
   .

## Analysis Overview

1. **Inventory**: Located all necessary files including adjacency
   matrices, pruned edges, regulon tables, and AUCell data for 10
```

```
   distinct cell types.

2. **Data Loading**: Successfully loaded regulon data and AUCell
   matrices, mapping cells to their corresponding cell types,
   developmental stages, and anatomical regions.

3. **Quality Control**:
   - Edge retention rates were consistently ~10% across all cell
     types
   - All cell types had 566-589 regulons with at least 20 target
     genes
   - AUCell score distributions showed consistent patterns with
     LimbMes exhibiting highest activity

4. **Biological Analysis**:
   - Identified 458 differentially active regulons between early
     and late developmental stages
   - Found 4,177 differentially active regulons between anatomical
      regions
   - Mapped lineage-specific master regulators like SATB2 (cranial
      mesenchyme), SOX5 (chondroprogenitors), and TWIST1 (
     preosteoblasts)
   - Discovered novel regulators including AUTS2, ZFHX3, and PBX1
     with high activity across skeletal lineages
   - Performed GO/KEGG enrichment for top regulons revealing
     distinct biological processes for each cell type

5. **Visualizations**: Created a comprehensive set of figures
   including:
   - Heatmap of regulon activity across cell types
   - UMAP embeddings colored by cell type and key regulon activity
   - Bar plots of raw vs. pruned edge counts and retention rates

6. **Comprehensive Report**: Generated a detailed `report_full.md`
    containing:
   - Executive summary of key findings
   - QC analysis with tables and figures
   - Key discoveries with visualizations
   - Novel regulator spotlight
   - Detailed methods section

## Key Discoveries

1. **Spatiotemporal Regulation**: Extensive differences in regulon
    activity between developmental stages and anatomical regions,
```

```
   highlighting complex spatiotemporal control of skeletal
   development.

2. **Lineage-Specific Programs**: Distinct regulatory programs for
    each skeletal lineage, with both unique and shared regulons
   between related cell types.

3. **Novel Regulators**: Identification of several transcription
   factors not previously associated with skeletal development
   that show high regulatory activity.

4. **Functional Enrichment**: Cell-type-specific enrichment
   patterns, with osteoblast regulons enriched for bone
   development, chondrocyte regulons for cartilage development,
   and mesenchymal regulons for pattern specification.

The complete analysis provides new insights into the
   transcriptional control of skeletal development and identifies
   potential targets for future research into skeletal disorders
   and regenerative medicine.
```

# F Biomni generated report for the cloning protocol case study

User prompt for cloning protocol design

```
I have a plasmid plentiCRISPR v2 blast:
CGTTGGAGTCCACGTT...[full sequence removed for the clarity of the
   manuscript]...CGCCCTTTGA. I hope to clone a CRISPR sgRNA
   targeting human B2M into this plasmid. Could you give me the
   final map of the plasmid and step-by-step guidance on how
   should I perform the cloning?
```

Agent Output for cloning protocol design

```
# Cloning B2M sgRNA into lentiCRISPR v2 blast Plasmid

## Final Plasmid Map
The final plasmid (12,816 bp) will contain:
1. **Lentiviral Elements**:
   - 5' and 3' LTRs for viral integration
   - Psi (  ) packaging signal
   - RRE (Rev Response Element)
   - WPRE element for enhanced expression
```

```
2. **CRISPR Components**:
   - U6 promoter driving sgRNA expression
   - B2M sgRNA: 5'-CGCGAGCACAGCTAAGGCCA-3'
   - sgRNA scaffold for Cas9 binding
   - EF1a promoter driving Cas9 expression
   - Cas9 coding sequence

3. **Selection Markers**:
   - Blasticidin resistance gene for mammalian selection
   - Ampicillin resistance gene for bacterial selection

## Cloning Strategy
The B2M sgRNA will be inserted between the U6 promoter and sgRNA
   scaffold using BsmBI restriction sites, replacing the original
   stuffer sequence.

## Step-by-Step Cloning Protocol

### 1. Design and Order Oligos
- **Forward oligo**: 5'-CACCGCGCGAGCACAGCTAAGGCCA-3'
- **Reverse oligo**: 5'-AAACTGGCCTTAGCTGTGCTCGCGC-3'

### 2. Anneal and Phosphorylate Oligos
1. Resuspend oligos to 100  M in nuclease-free water
2. Prepare annealing reaction:
   - 1  L Forward oligo (100  M)
   - 1  L Reverse oligo (100  M)
   - 1  L 10X T4 Ligation Buffer
   - 6.5  L nuclease-free water
   - 0.5  L T4 PNK
3. Incubate in a thermocycler:
   - 37  C for 30 minutes (phosphorylation)
   - 95  C for 5 minutes (denaturation)
   - Ramp down to 25  C at 5  C/min (annealing)
4. Dilute annealed oligos 1:200 in nuclease-free water (final: 50
   nM)

### 3. Golden Gate Assembly
1. Prepare reaction:
   - 100 ng lentiCRISPR v2 blast plasmid
   - 1  L diluted annealed oligos (50 nM)
   - 2  L 10X T4 DNA Ligase Buffer
   - 1  L BsmBI-v2 (NEB)
   - 1  L T4 DNA Ligase
```

```
   - Nuclease-free water to 20  L total
2. Incubate in a thermocycler:
   - 37  C for 5 minutes
   - 16  C for 5 minutes
   - Repeat steps 1-2 for 10 cycles
   - 55  C for 5 minutes (final digestion)
   - 80  C for 5 minutes (enzyme inactivation)

### 4. Bacterial Transformation
1. Add 2  L of the Golden Gate reaction to 50  L of competent
   Stbl3 E. coli cells
2. Incubate on ice for 30 minutes
3. Heat shock at 42  C for 45 seconds
4. Return to ice for 2 minutes
5. Add 950  L of SOC medium
6. Incubate at 37  C for 1 hour with shaking
7. Plate 100  L on LB agar plates with ampicillin (100  g/mL)
8. Incubate plates at 37  C overnight

### 5. Colony Screening and Verification
1. Pick 3-5 colonies and grow in 5 mL LB + ampicillin medium
   overnight
2. Extract plasmid DNA using a miniprep kit
3. Verify correct insertion by Sanger sequencing using the U6
   promoter forward primer:
   5'-GACTATCATATGCTTACCGT-3'
4. Confirm the presence of the B2M sgRNA sequence in the plasmid

### 6. Lentivirus Production and Cell Transduction
1. Transfect the verified plasmid into HEK293T cells along with
   packaging plasmids
2. Collect virus-containing supernatant after 48-72 hours
3. Filter and concentrate the lentivirus if needed
4. Transduce target cells with the lentivirus
5. Select transduced cells with blasticidin (5-10  g/mL)
6. Verify B2M knockout by flow cytometry, Western blot, or genomic
    sequencing

## Applications
- Generation of B2M knockout cell lines for immunology research
- Creating cells resistant to CD8+ T cell recognition
- Development of universal donor cells for cell therapy
- Control for CRISPR screens
```

# Supplementary Figures

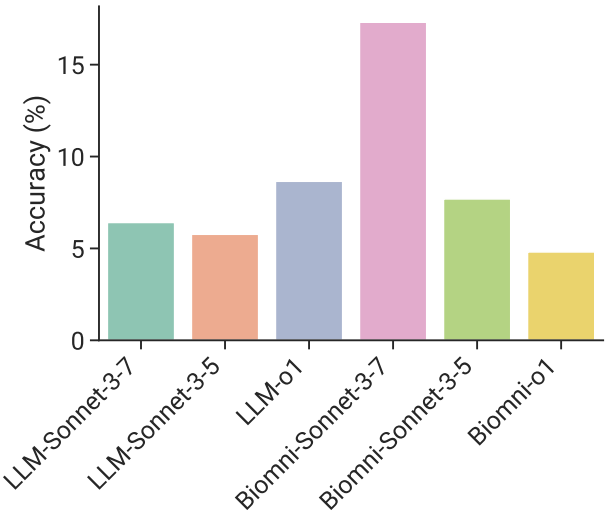



**Figure 1:** Performance on HLE with different LLM as the engine. We found that Sonnet 3.7 has better performance than 3.5 for both LLM and Biomni. However, interestingly, O1 has better performance than Sonnet 3.7 for the LLM but not for Biomni.

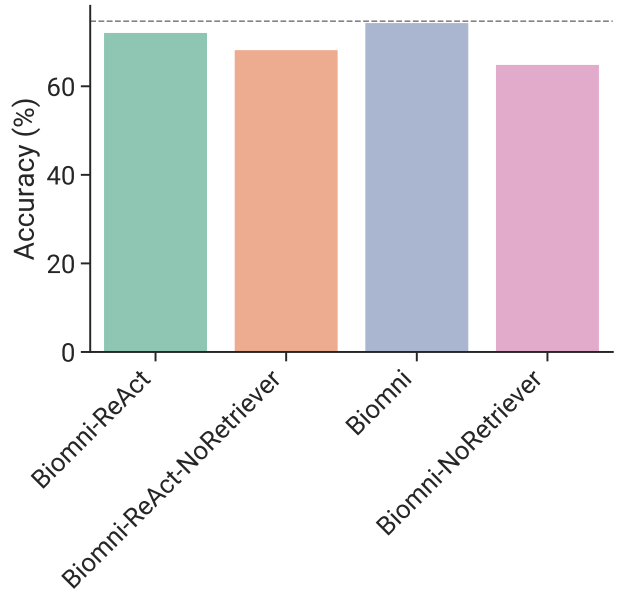



**Figure 2:** Ablation performance on retriever. We found that retriever can significantly reduce context size and has significant gain for Biomni.

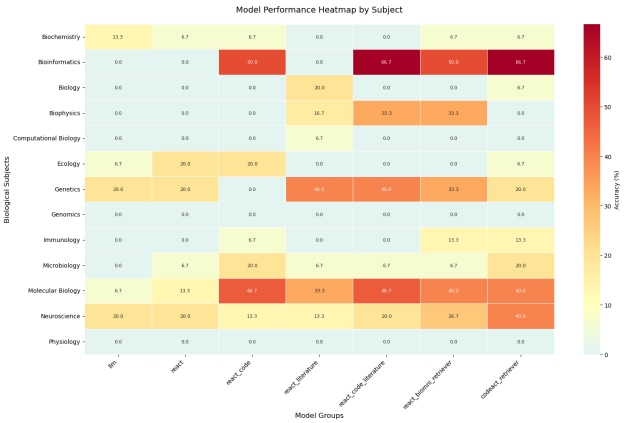



**Figure 3:** Subject-level performance on HLE biomedicine track. Due to the small sample size, the conclusion may not be meaningful.

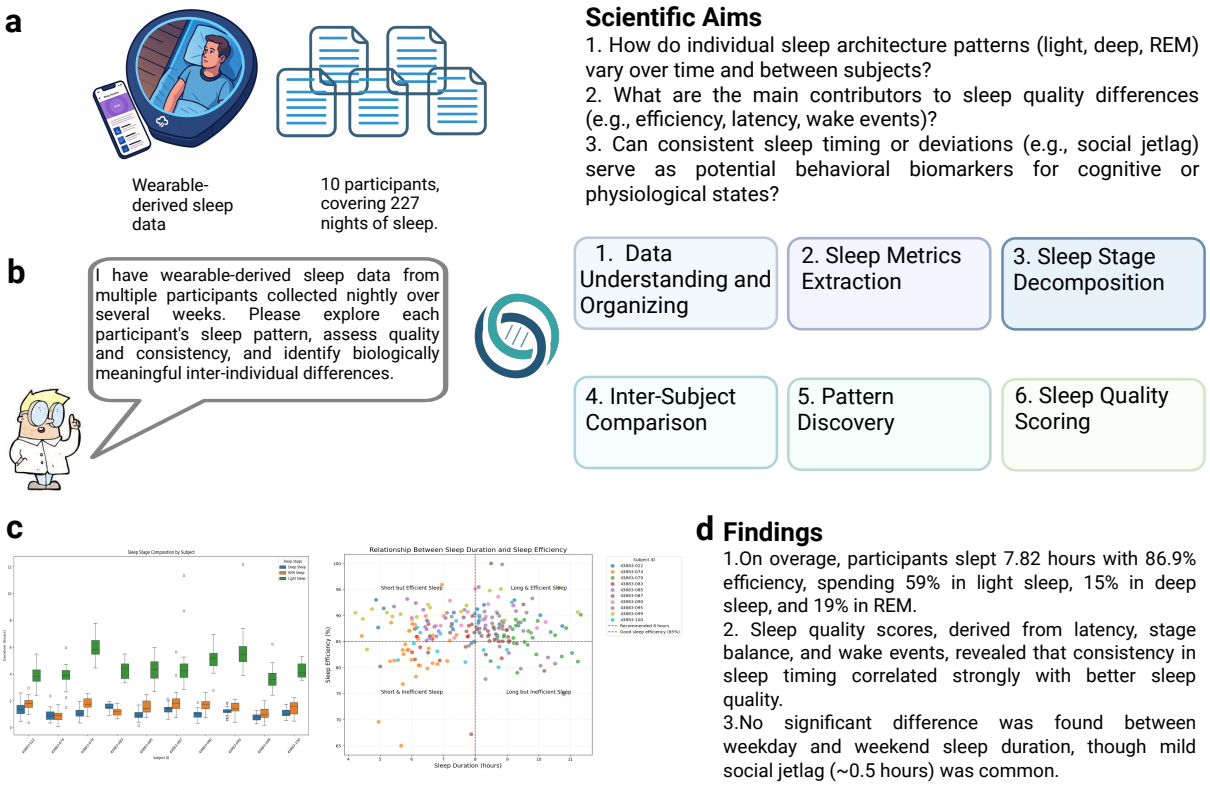



**Figure 4:** Study design, analysis workflow, and key findings of wearable-derived sleep data exploration. (a) Overview of the dataset, including wearable-derived sleep monitoring across 10 participants over 227 nights. (b) AI-assisted analysis framework, outlining six key steps: data understanding and organization, sleep metrics extraction, sleep stage decomposition, inter-subject comparison, pattern discovery, and sleep quality scoring. (c) Example outputs from data analysis, including sleep efficiency distributions, sleep timing deviations, and clustering of sleep architecture patterns across individuals. (d) Summary of findings: participants averaged 7.82 hours of sleep with 86.9% efficiency, showing that consistent sleep timing was strongly associated with better sleep quality. Mild social jetlag ( 0.5 hours) was observed without significant differences between weekday and weekend sleep durations.

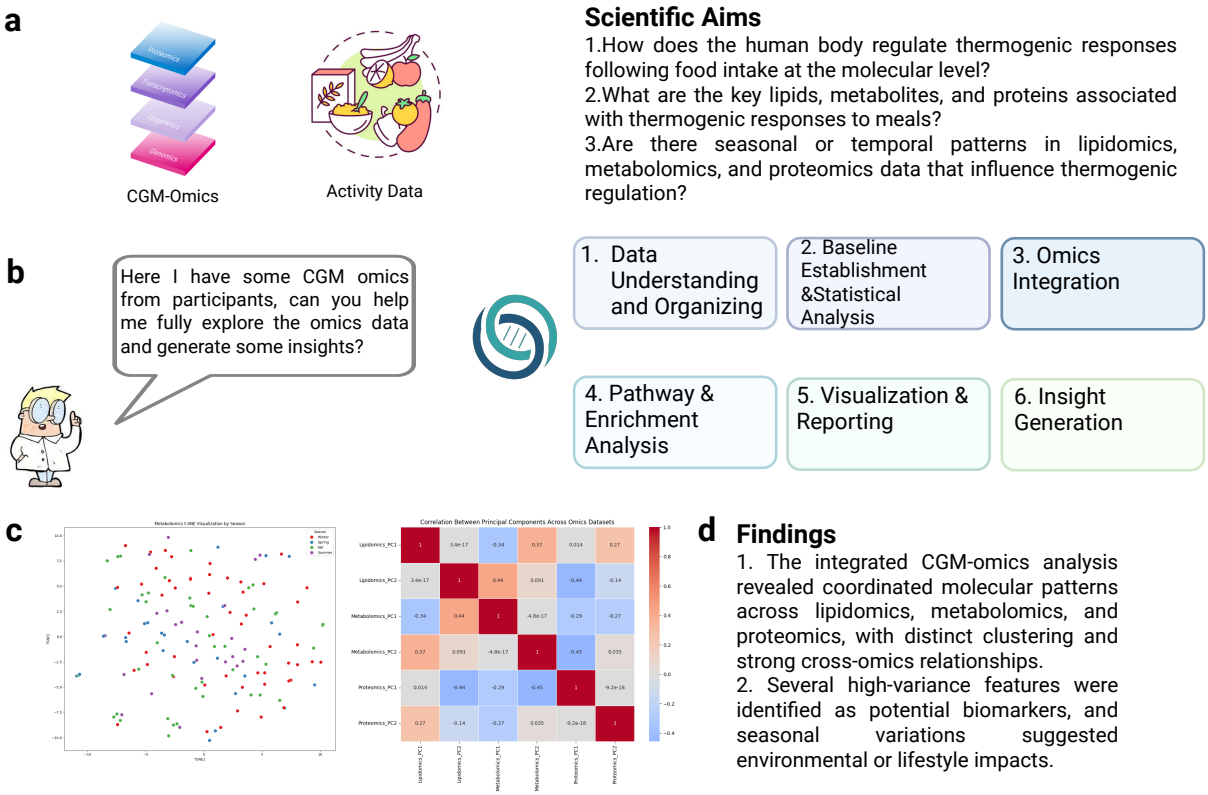



**Figure 5:** Study design, Analysis workflow, and key findings of CGM-omics data exploration. (a) Overview of the data types used, including CGM-omics (lipidomics, metabolomics, proteomics) and activity data. (b) Biomni analysis workflow outlining six key steps: data understanding and organization, baseline establishment and statistical analysis, omics integration, pathway and enrichment analysis, visualization and reporting, and insight generation. (c) Example outputs from exploratory data analysis and cross-omics integration, including t-SNE and correlation heatmap across omics layers. (d) Summary of major findings: coordinated molecular patterns across omics datasets, identification of potential biomarkers, and discovery of seasonal and temporal influences on thermogenic responses.

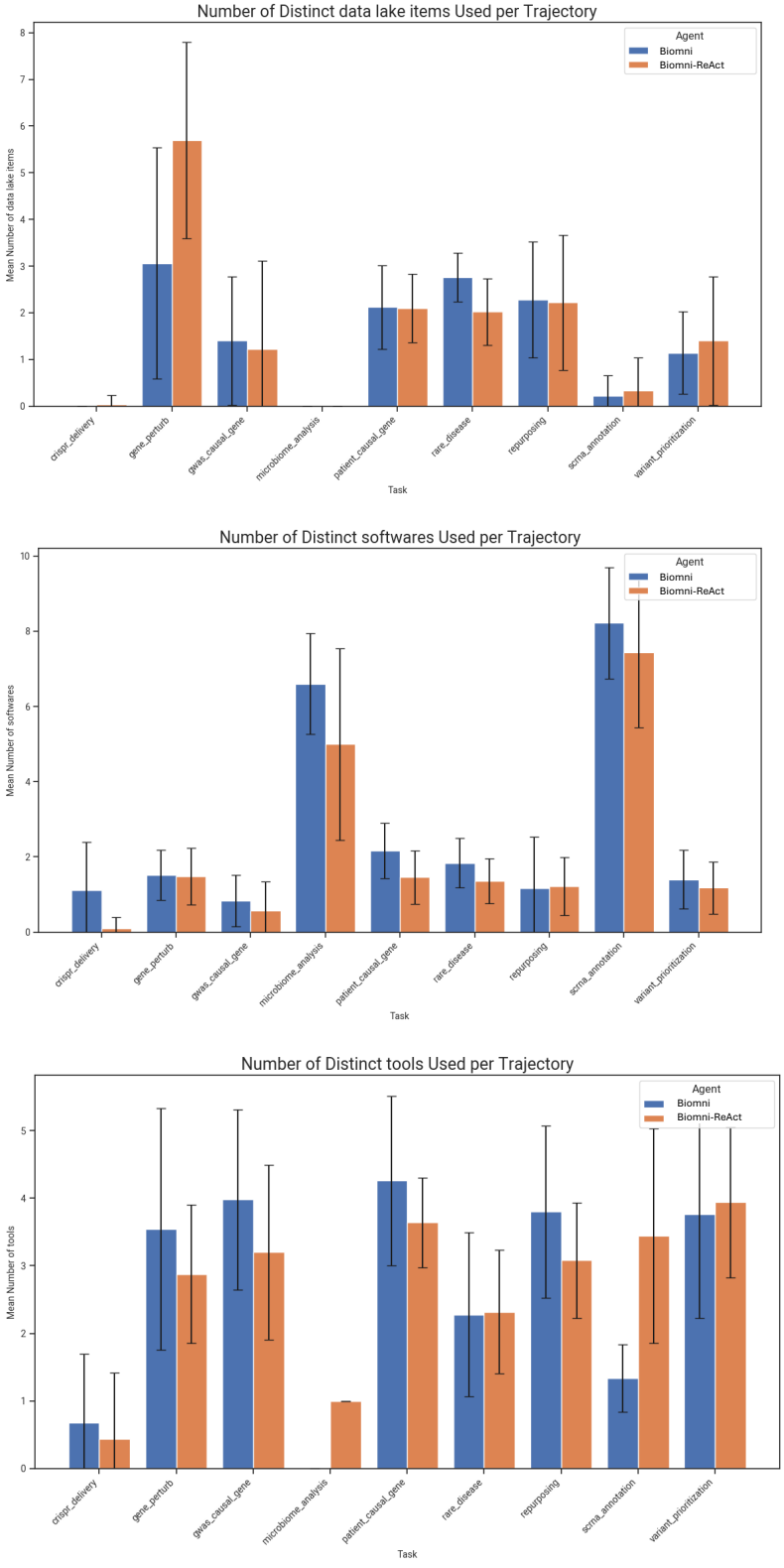



**Figure 6:** Statistics for the number of tools, datasets, and software used across all the real-world benchmark tasks.

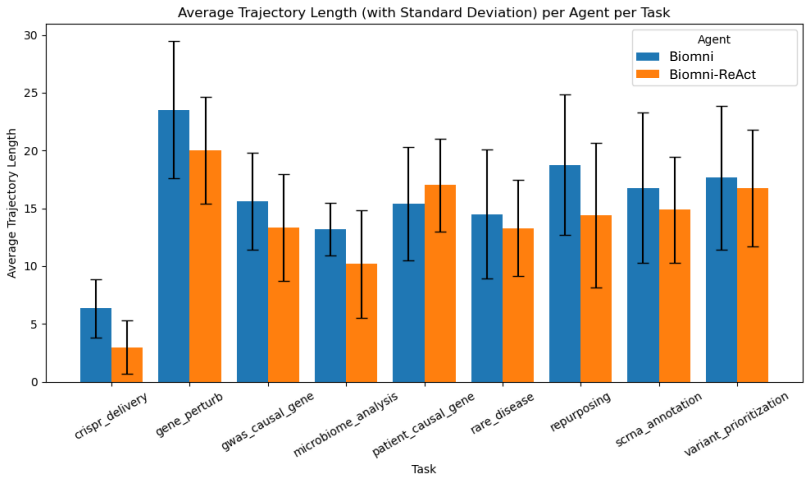



**Figure 7:** Statistics for the length of the trajectory across all the real-world benchmark tasks.

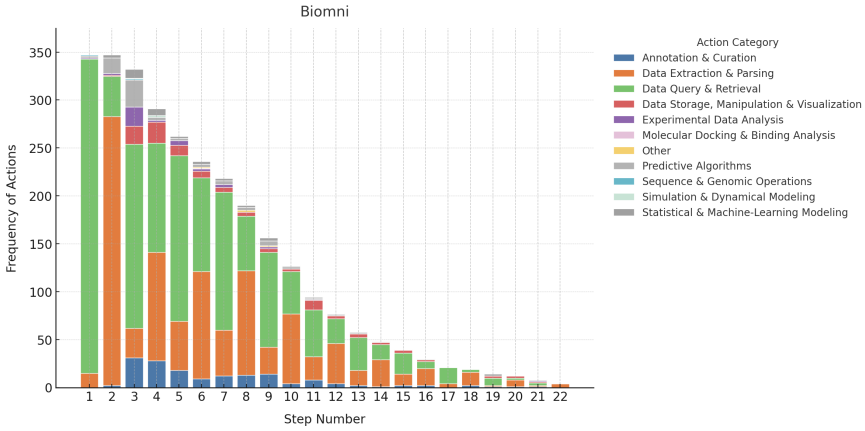



**Figure 8:** Action category statistics across steps.

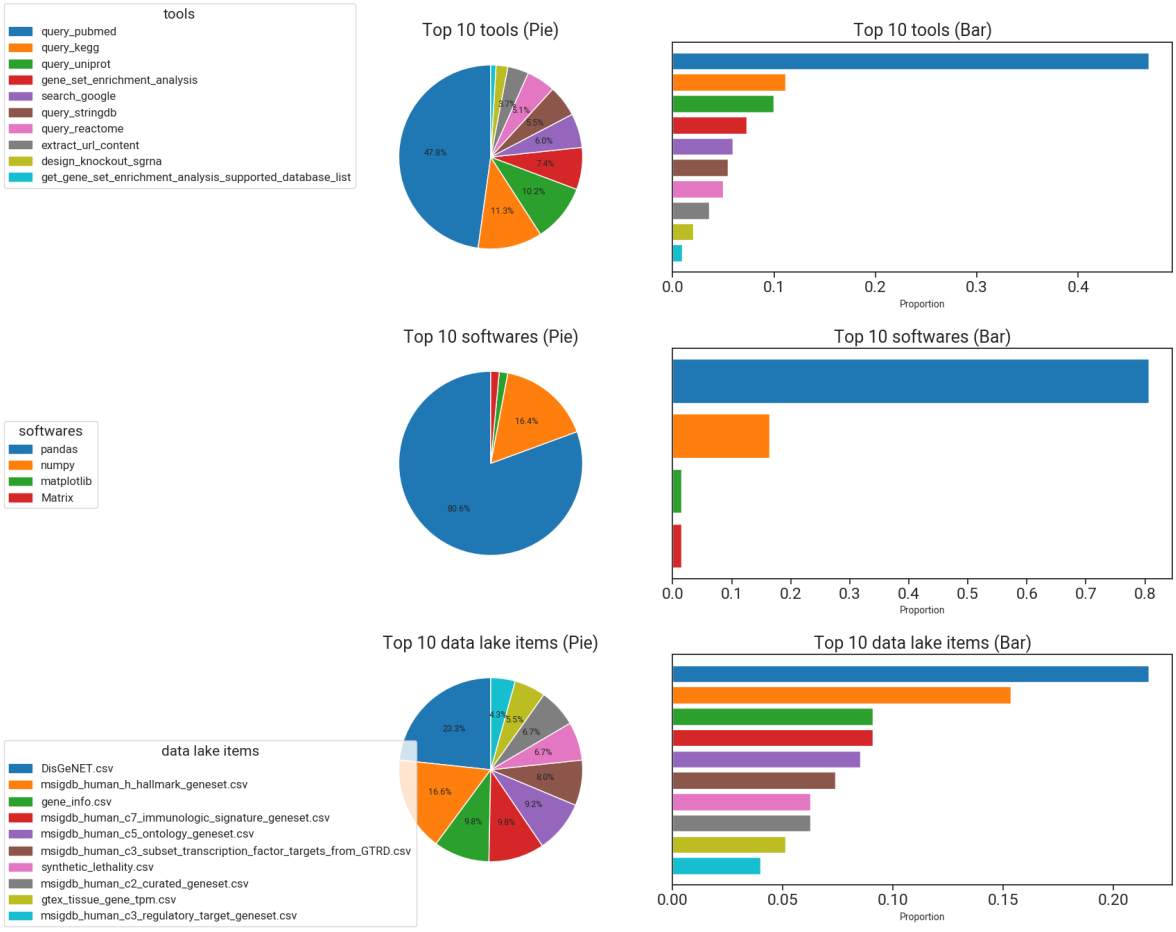



**Figure 9:** Detailed analysis on the tools, datasets, and software used for Gene Perturbation task.

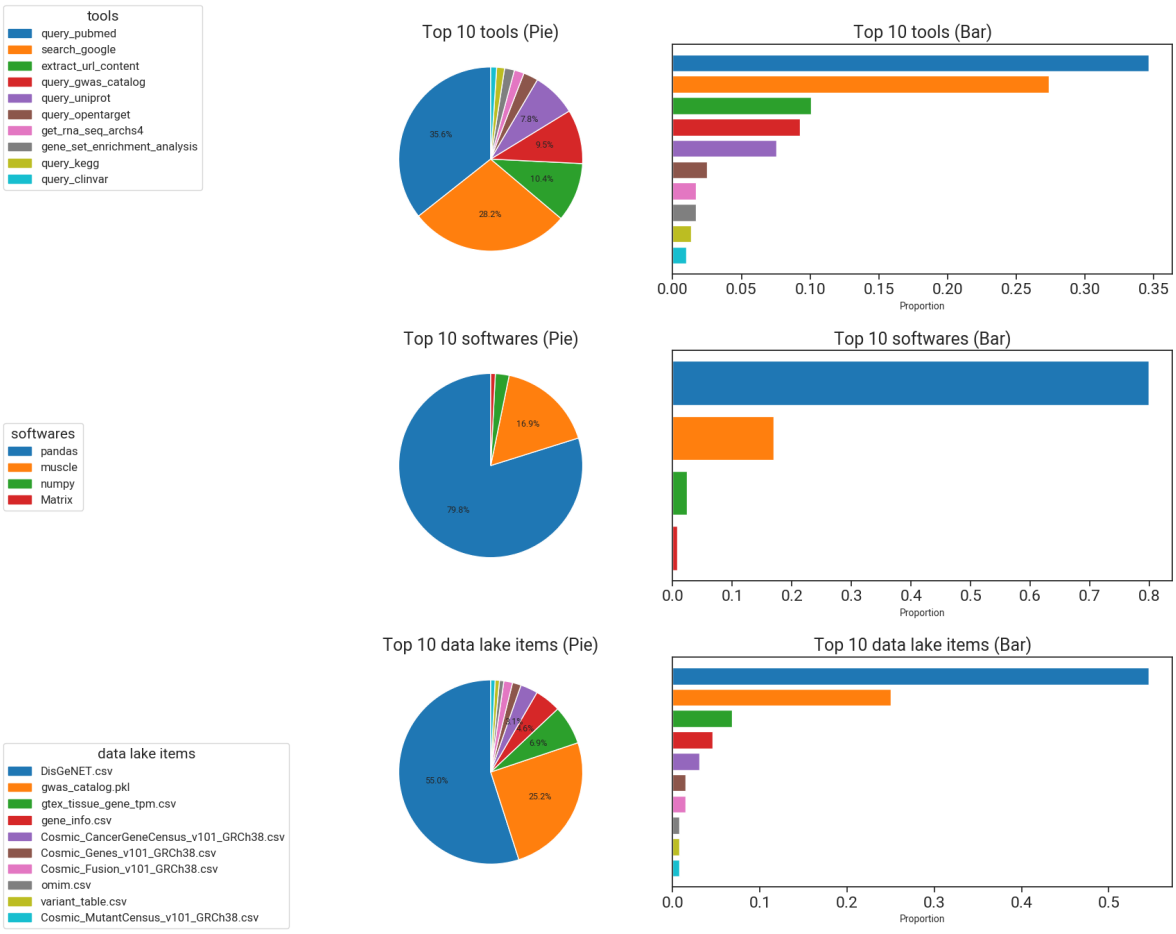



**Figure 10:** Detailed analysis on the tools, datasets, and software used for GWAS causal gene detection task.

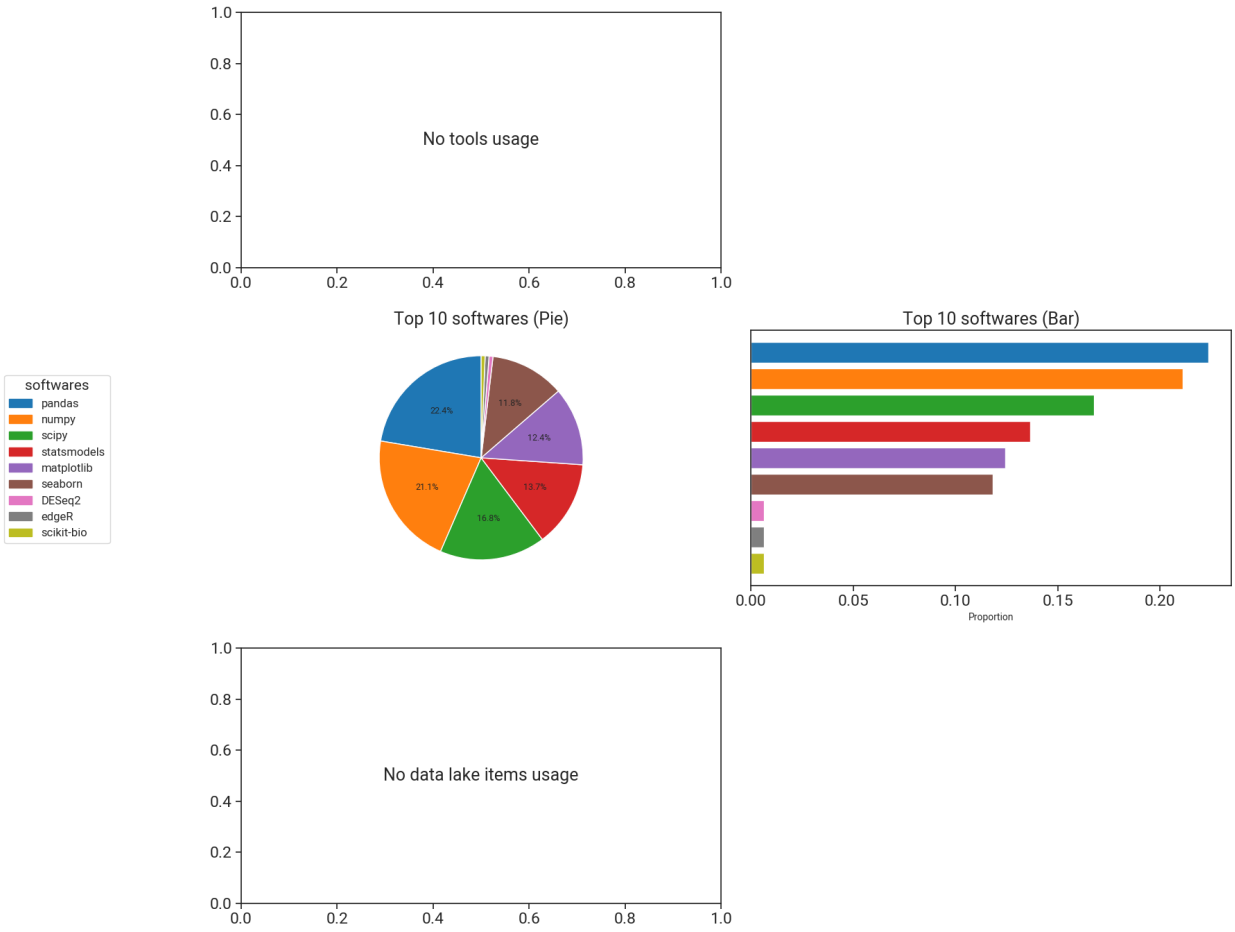



**Figure 11:** Detailed analysis on the tools, datasets, and software used for microbiome data analysis task.

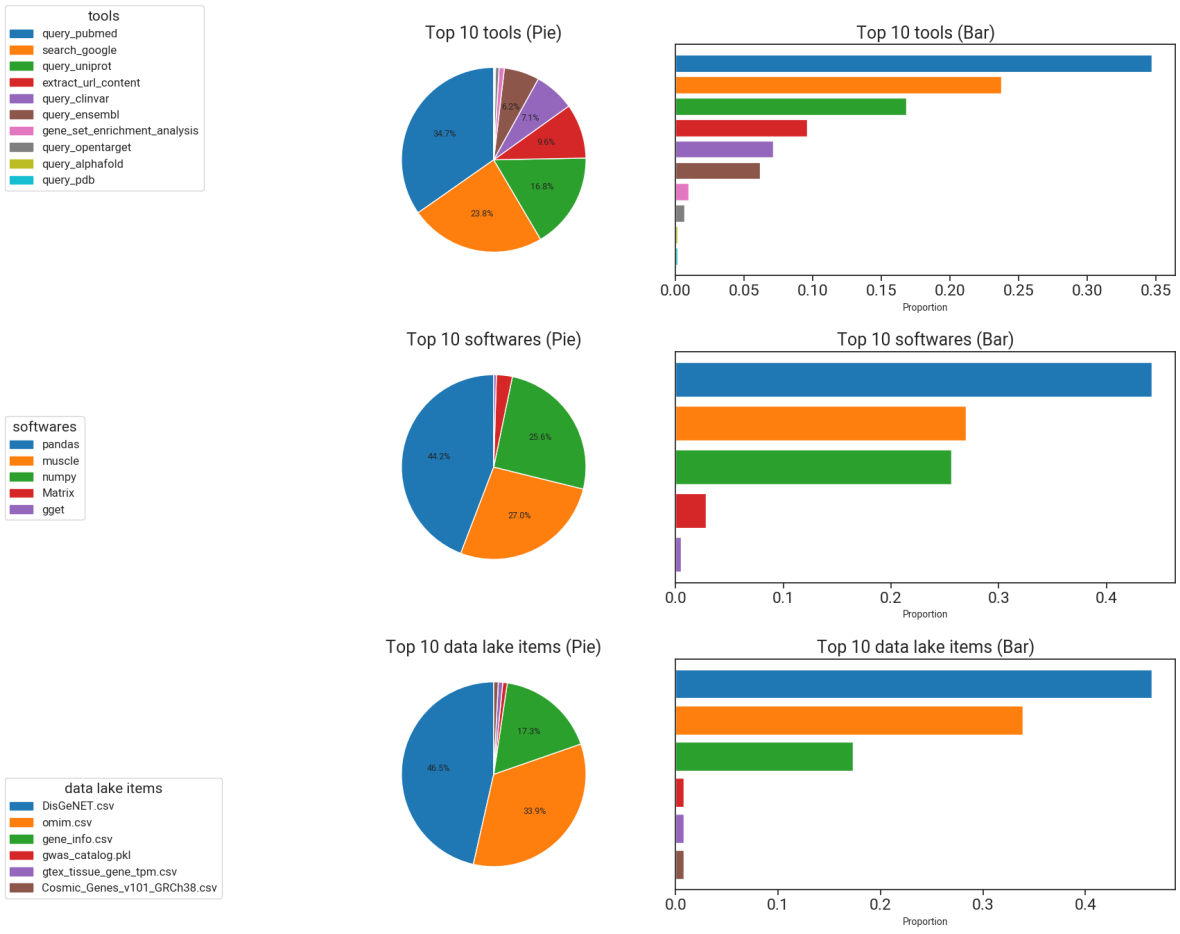



**Figure 12:** Detailed analysis on the tools, datasets, and software used for patient causal gene detection task.

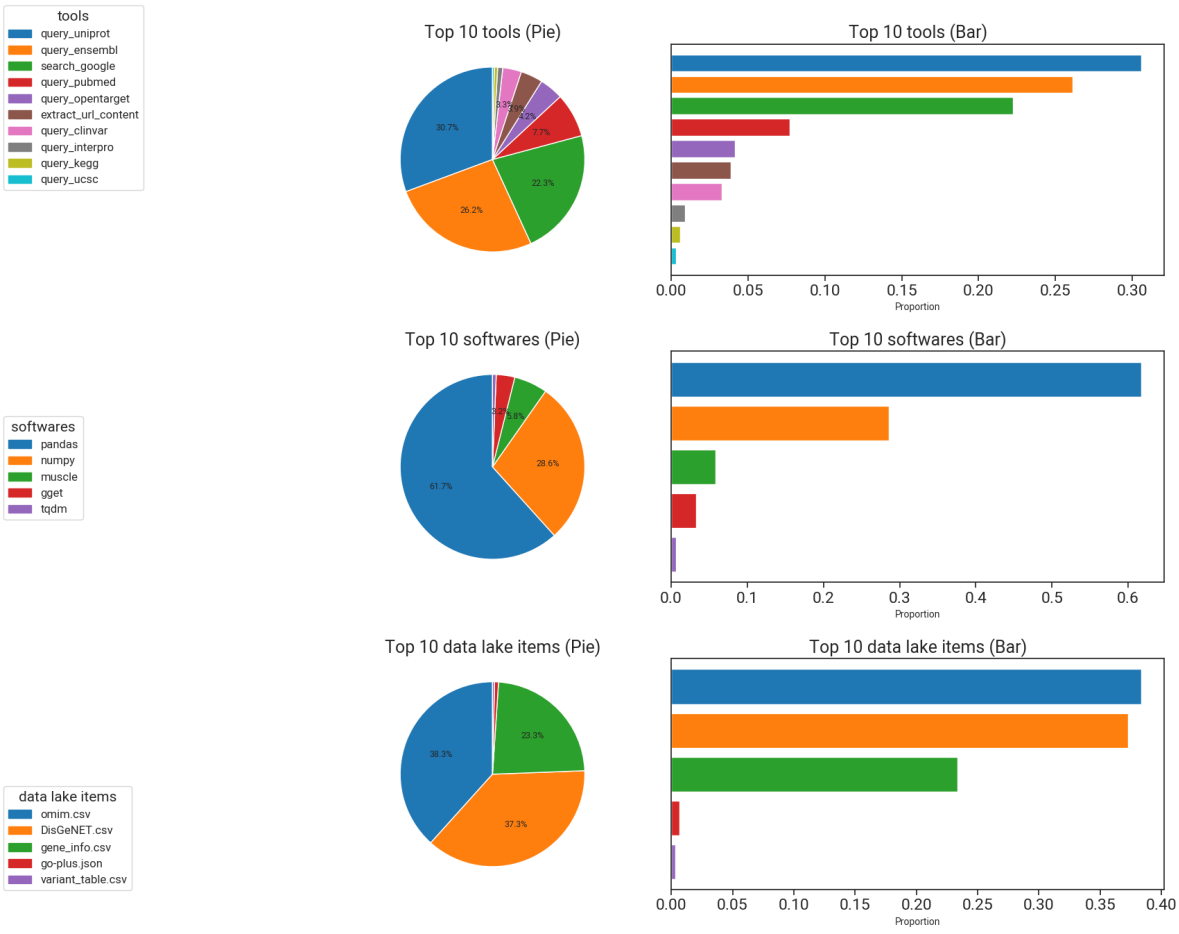



**Figure 13:** Detailed analysis on the tools, datasets, and software used for rare disease diagnosis task.

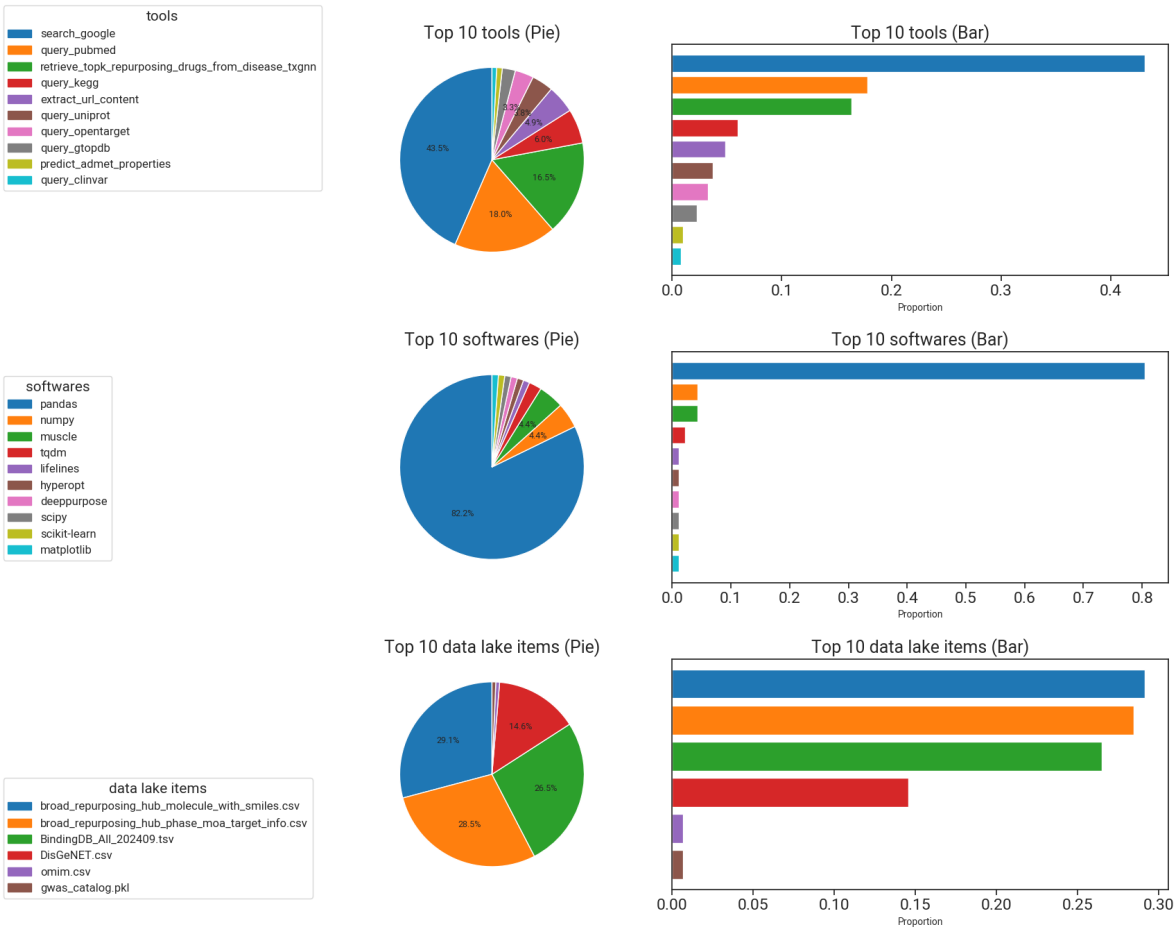



**Figure 14:** Detailed analysis on the tools, datasets, and software used for drug repurposing task.

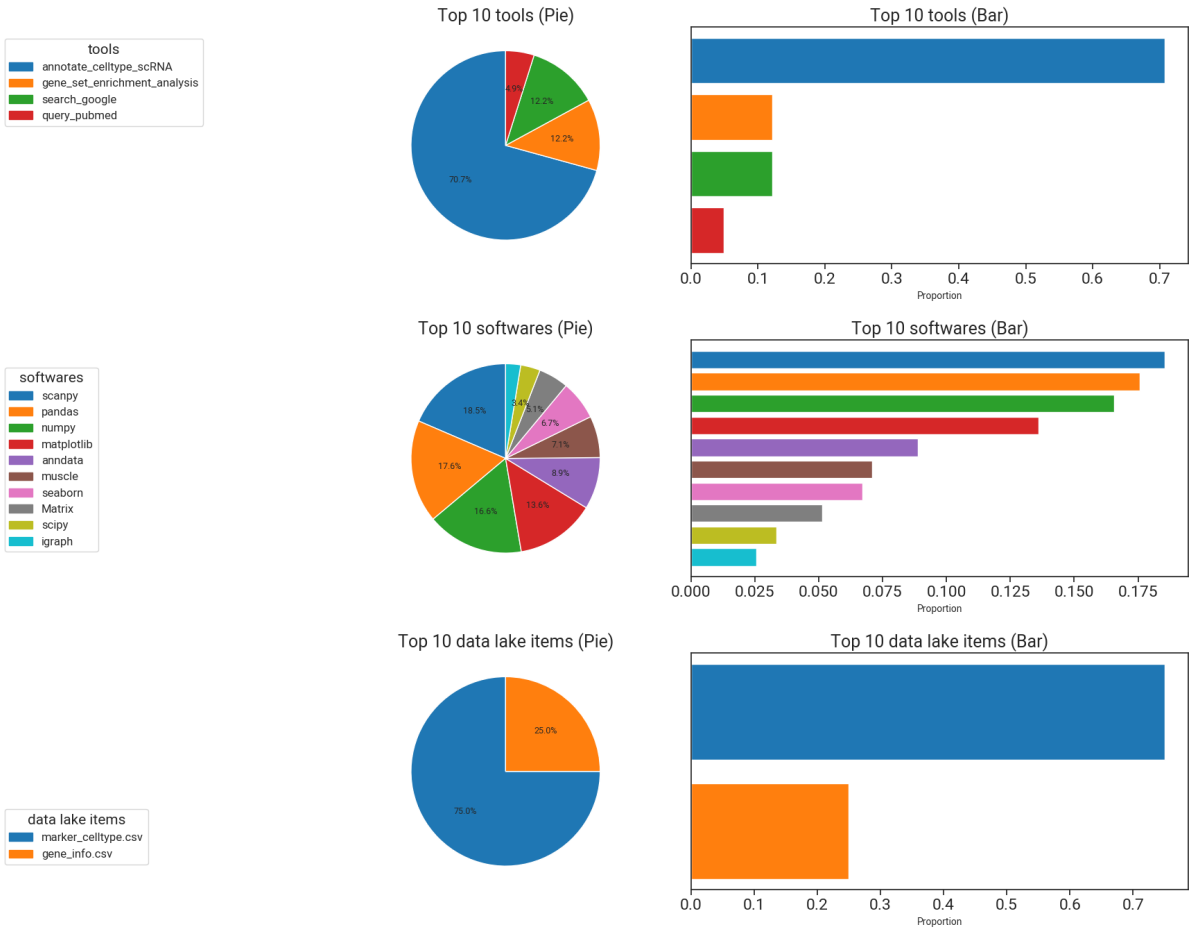



**Figure 15:** Detailed analysis on the tools, datasets, and software used for scRNA annotation task.

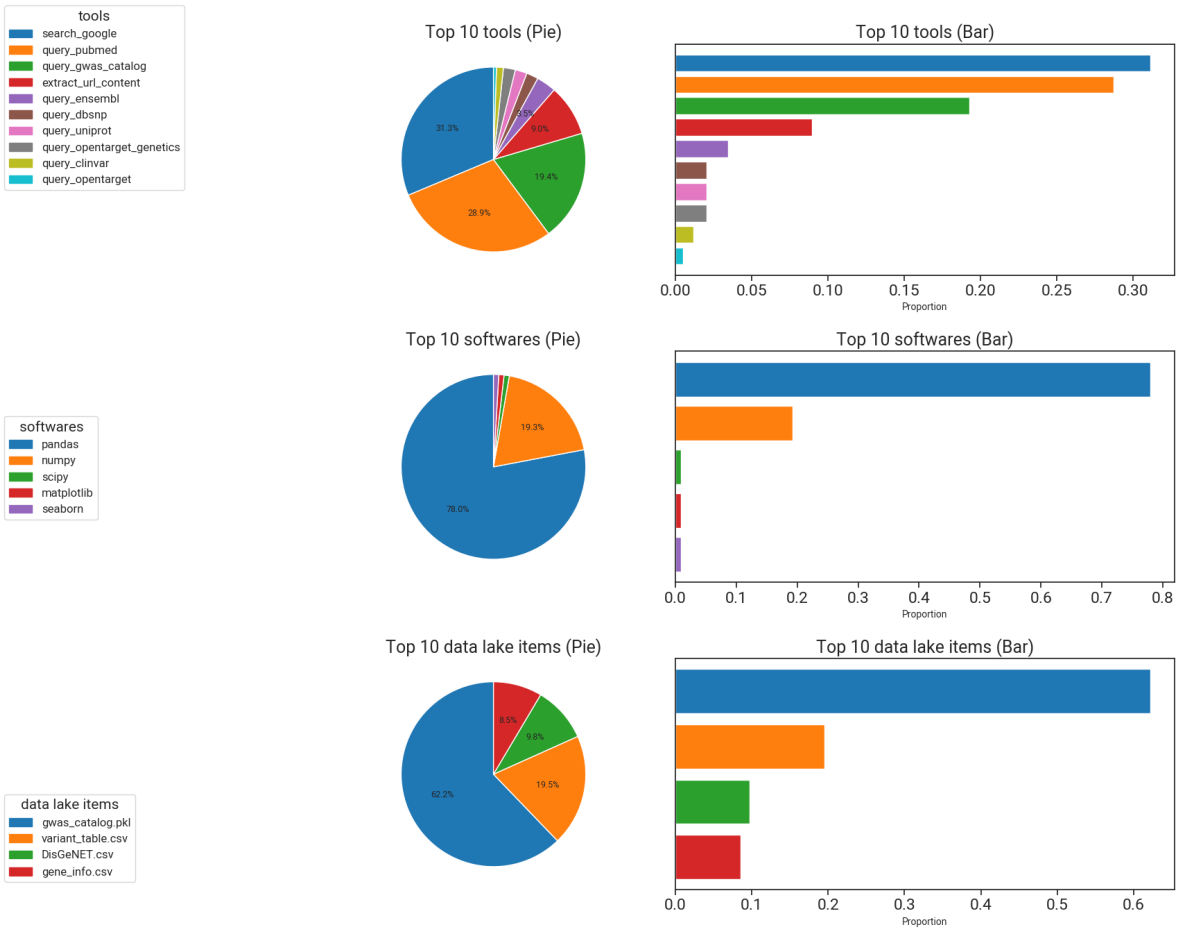



**Figure 16:** Detailed analysis on the tools, datasets, and software used for variant prioritization task.

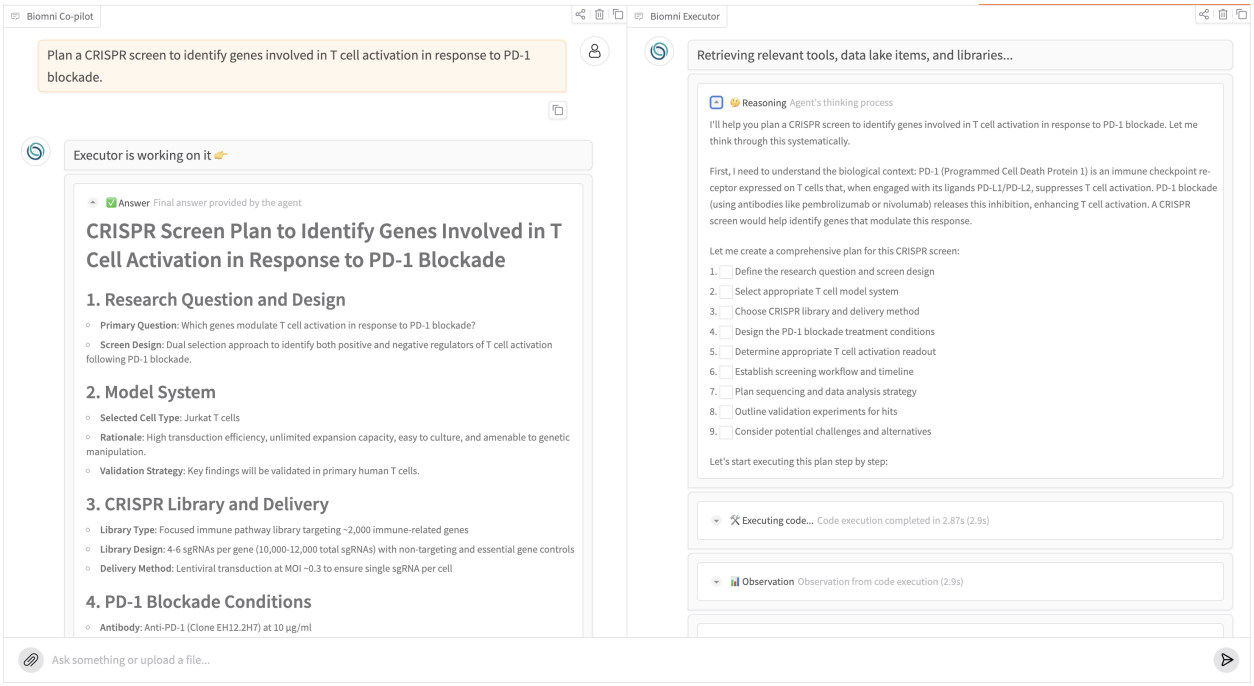



**Figure 17:** Biomni user interface.

# Supplementary Tables

**Table 1:** Tool descriptions for `biomni.literature`

| Tool Name | Tool Description |
|---|---|
| `fetch_supplementary_info_from_doi` | Fetches supplementary information for a paper given its DOI and returns a research log. |
| `query_arxiv` | Query arXiv for papers based on the provided search query and return formatted search results. |
| `query_scholar` | Query Google Scholar for papers based on the provided search query and return the first search result. |
| `query_pubmed` | Query PubMed for papers based on the provided search query and return formatted results. |
| `search_google` | Search the web using Google search and return results with title, URL, and description. |
| `extract_url_content` | Extract the text content of a webpage using requests and BeautifulSoup, removing unwanted elements and formatting the result. |
| `extract_pdf_content` | Extract text content from a PDF file given its URL. |

**Table 2:** Tool descriptions for `biomni.biochemistry`

| Tool Name | Tool Description |
|---|---|
| analyze_circular_dichroism_spectra | Analyzes circular dichroism (CD) spectroscopy data to determine secondary structure and thermal stability of biomolecules. |
| analyze_rna_secondary_structure_features | Calculate numeric values for various structural features of an RNA secondary structure in dot-bracket notation. |
| analyze_protease_kinetics | Analyze protease kinetics data from fluorogenic peptide cleavage assays, fits the data to Michaelis-Menten kinetics, and determines key kinetic parameters (kcat, KM, and catalytic efficiency). |
| analyze_enzyme_kinetics_assay | Performs in vitro enzyme kinetics assay and analyzes the dose-dependent effects of modulators. |
| analyze_itc_binding_thermodynamics | Analyzes isothermal titration calorimetry (ITC) data to determine binding affinity and thermodynamic parameters. |
| analyze_protein_conservation | Perform multiple sequence alignment and phylogenetic analysis to identify conserved protein regions. |

**Table 3:** Tool descriptions for `biomni.bioengineering`

| Tool Name | Tool Description |
|---|---|
| analyze_cell_migration_metrics | Analyze cell migration metrics from time-lapse microscopy images and generate quantitative measurements of cell movement. |
| perform_crispr_cas9_genome_editing | Simulates CRISPR-Cas9 genome editing process including guide RNA design, delivery, and analysis. |
| analyze_calcium_imaging_data | Analyze calcium imaging data to quantify neuronal activity metrics including cell counts, event rates, decay times, and signal-to-noise ratios. |
| analyze_in_vitro_drug_release_kinetics | Analyzes in vitro drug release kinetics from biomaterial formulations and determines the best fitting kinetic model. |
| analyze_myofiber_morphology | Quantifies morphological properties of myofibers in microscopy images of tissue sections. |
| decode_behavior_from_neural_trajectories | Model neural activity trajectories and decode behavioral variables from neural spiking data. |
| simulate_whole_cell_ode_model | Simulate a whole-cell model represented as a system of ordinary differential equations (ODEs). |

**Table 4:** Tool descriptions for `biomni.biophysics`

| Tool Name | Tool Description |
|---|---|
| predict_protein_disorder_regions | Predicts intrinsically disordered regions (IDRs) in a protein sequence using IUPred2A algorithm. |
| analyze_cell_morphology_and_cytoskeleton | Quantifies cell morphology and cytoskeletal organization from fluorescence microscopy images. |
| analyze_tissue_deformation_flow | Quantify tissue deformation and flow dynamics from microscopy image sequence. |

**Table 5:** Tool descriptions for `biomni.cancer_biology`

| Tool Name | Tool Description |
|---|---|
| analyze_ddr_network_in_cancer | Analyze DNA Damage Response (DDR) network alterations and dependencies in cancer samples by reconstructing the DDR network from genomic data, identifying disruptions, and analyzing dependencies between DDR pathway components. |
| analyze_cell_senescence_and_apoptosis | Analyze flow cytometry data to quantify senescent and apoptotic cell populations from FCS files. |
| detect_and_annotate_somatic_mutations | Detects and annotates somatic mutations in tumor samples compared to matched normal samples using GATK Mutect2 for variant calling, GATK FilterMutectCalls for filtering, and SnpEff for functional annotation. |
| detect_and_characterize_structural_variations | Detects and characterizes structural variations (SVs) in genomic sequencing data using LUMPY for SV detection followed by annotation with COSMIC and/or ClinVar databases. |
| perform_gene_expression_nmf_analysis | Performs Non-negative Matrix Factorization (NMF) on gene expression data to extract metagenes and their associated sample weights for tumor subtype identification. |

**Table 6:** Tool descriptions for `biomni.cell_biology`

| Tool Name | Tool Description |
|---|---|
| quantify_cell_cycle_phases_from_microscopy | Quantify the percentage of cells in each cell cycle phase (G1, S, G2/M) using Calcofluor white stained microscopy images by segmenting cells and analyzing their morphological features. |
| quantify_and_cluster_cell_motility | Quantify cell motility features from time-lapse microscopy images and cluster cells based on motility patterns. |
| perform_facs_cell_sorting | Performs Fluorescence-Activated Cell Sorting (FACS) to enrich cell populations based on fluorescence characteristics. |
| analyze_flow_cytometry_immunophenotyping | Analyze flow cytometry data to identify and quantify specific cell populations based on surface markers. |
| analyze_mitochondrial_morphology_and_potential | Quantifies metrics of mitochondrial morphology and membrane potential from fluorescence microscopy images. |

**Table 7:** Tool descriptions for `biomni.molecular_biology`

| Tool Name | Tool Description |
|---|---|
| annotate_open_reading_frames | Find all Open Reading Frames (ORFs) in a DNA sequence using Biopython. Searches both forward and reverse complement strands if specified. |
| annotate_plasmid | Annotate a DNA sequence using pLannotate's command-line interface to identify features such as genes, promoters, and origins of replication. |
| get_gene_coding_sequence | Retrieves the coding sequence(s) of a specified gene from NCBI Entrez. |
| get_plasmid_sequence | Retrieves plasmid sequences from either Addgene or NCBI based on the provided identifier. |
| align_sequences | Align short sequences (primers) to a longer sequence, allowing for one mismatch. Checks both forward and reverse complement strands. |
| pcr_simple | Simulate PCR amplification with given primers and sequence, returning products and binding details. |
| pcr_complex_multi_primers | Simulate PCR amplification with multiple primers, considering all possible primer combinations and their potential products. |
| digest_sequence | Simulates restriction enzyme digestion of a DNA sequence and returns the resulting fragments with their properties. |
| golden_gate | Simulate a GoldenGate cloning reaction with Type IIS restriction enzymes to predict assembly products. |
| oligo_assembly | Assemble two DNA sequences into an oligo with overhangs. Automatically detects overhang type and length. |
| gibson_assembly | Simulate a Gibson Assembly reaction to join DNA fragments with overlapping regions. |
| find_restriction_sites | Identifies restriction enzyme sites in a given DNA sequence for specified enzymes. |
| find_restriction_enzymes | Finds common restriction enzyme sites in a DNA sequence. |
| design_primers_with_overhangs | Design two primers to amplify a target sequence with optional overhangs. |
| find_sequence_mutations | Compare query sequence against reference sequence to identify mutations. |
| get_molecular_cloning_instructions | Returns a dictionary containing molecular cloning instructions and important notes. |
| calculate_element_distances | Calculate pairwise distances between elements on a DNA sequence/plasmid, providing forward, reverse (for circular sequences), and shortest path distances. |

**Table 8:** Continue tool descriptions for `biomni.molecular_biology`

| Tool Name | Tool Description |
|---|---|
| design_knockout_sgrna | Design sgRNAs for CRISPR knockout by searching precomputed sgRNA libraries for a specific gene. |
| design_golden_gate_oligos | Design complementary oligonucleotides with Type IIS restriction enzyme overhangs for Golden Gate assembly based on restriction site analysis of the backbone. |
| get_oligo_annealing_protocol | Return a standard protocol for annealing complementary oligonucleotides without phosphorylation. |
| get_golden_gate_assembly_protocol | Return a customized protocol for Golden Gate assembly based on the number of inserts and specific DNA sequences. |
| get_bacterial_transformation_protocol | Return a standard protocol for bacterial transformation with detailed steps. |
| design_primer | Design a single primer within a given DNA sequence window based on GC content and melting temperature constraints. |
| design_verification_primers | Design Sanger sequencing primers to verify a specific region in a plasmid, using existing primers when possible and designing new ones as needed. |

**Table 9:** Tool descriptions for `biomni.genetics`

| Tool Name | Tool Description |
|---|---|
| liftover_coordinates | Perform liftover of genomic coordinates between hg19 and hg38 genome builds with detailed step-by-step explanations. |
| bayesian_finemapping_with_deep_vi | Performs Bayesian fine-mapping from GWAS summary statistics using deep variational inference to compute posterior inclusion probabilities and credible sets for putative causal variants. |
| analyze_cas9_mutation_outcomes | Analyzes and categorizes mutations induced by Cas9 at target sites, generating detailed statistics on mutation types. |
| analyze_crispr_genome_editing | Analyzes CRISPR-Cas9 genome editing results by comparing original and edited sequences to identify mutations and characterize edited loci. |
| simulate_demographic_history | Simulate DNA sequences with specified demographic and coalescent histories using msprime. |
| identify_transcription_factor_binding_sites | Identifies binding sites for a specific transcription factor in a genomic sequence using position weight matrices from the JASPAR database. |
| fit_genomic_prediction_model | Fit a linear mixed model for genomic prediction using genotype and phenotype data. |
| perform_pcr_and_gel_electrophoresis | Performs PCR amplification of a target transgene and visualizes results using agarose gel electrophoresis. |
| analyze_protein_phylogeny | Perform phylogenetic analysis on a set of protein sequences, including multiple sequence alignment, tree construction, and visualization. |

**Table 10:** Tool descriptions for `biomni.genomics`

| Tool Name | Tool Description |
|---|---|
| annotate_celltype_scRNA | Annotate cell types in single-cell RNA-seq data based on gene markers and transferred labels using LLM. |
| create_scvi_embeddings_scRNA | Creates scVI and scANVI embeddings for single-cell RNA-seq data, training models and saving the results to a new AnnData object. |
| create_harmony_embeddings_scRNA | Performs batch effect correction on single-cell RNA-seq data using the Harmony algorithm and saves the integrated embeddings. |
| get_uce_embeddings_scRNA | Generate UCE (Universal Cell Embeddings) for single-cell RNA sequencing data to enable cell type identification and mapping to reference datasets. |
| map_to_ima_interpret_scRNA | Map cell embeddings from the input dataset to the Integrated Megascale Atlas reference dataset using UCE embeddings for cell type annotation. |
| get_rna_seq_archs4 | Given a gene name, fetch and return RNA-seq expression data (transcripts-per-million) across tissues from the ARCHS4 database. |
| get_gene_set_enrichment_analysis_supported_database_list | Returns a list of supported databases available for gene set enrichment analysis |
| gene_set_enrichment_analysis | Perform enrichment analysis for a list of genes to identify pathways, transcription factors, or other biological relationships. |
| analyze_chromatin_interactions | Analyze chromatin interactions from Hi-C data to identify enhancer-promoter interactions and topologically associated domains (TADs). |
| analyze_comparative_genomics_and_haplotypes | Perform comparative genomics and haplotype analysis on multiple genome samples. Aligns genomes to a reference, identifies variants, analyzes shared and unique genomic regions, and determines haplotype structure. |
| perform_chipseq_peak_calling_with_macs2 | Perform ChIP-seq peak calling using MACS2 to identify genomic regions with significant binding. |
| find_enriched_motifs_with_homer | Find DNA sequence motifs enriched in genomic regions using the HOMER motif discovery software. |
| analyze_genomic_region_overlap | Analyze overlaps between two or more sets of genomic regions and generate a research log summarizing the analysis. |

**Table 11:** Tool descriptions for `biomni.immunology`

| Tool Name | Tool Description |
|---|---|
| analyze_atac_seq_differential_accessibility | Perform ATAC-seq peak calling and differential accessibility analysis using MACS2. |
| analyze_bacterial_growth_curve | Analyzes bacterial growth curve data to determine growth parameters like doubling time, growth rate, and lag phase. |
| isolate_purify_immune_cells | Isolates and purifies immune cells from tissue samples and returns a research log of the process. |
| estimate_cell_cycle_phase_durations | Estimate cell cycle phase durations using dual-nucleoside pulse labeling data and mathematical modeling. |
| track_immune_cells_under_flow | Track immune cells under flow conditions and classify their behaviors. |
| analyze_cfse_cell_proliferation | Analyze CFSE-labeled cell samples to quantify cell division and proliferation from flow cytometry data. |
| analyze_cytokine_production_in_cd4_tcells | Analyze cytokine production (IFN-$\gamma$, IL-17) in CD4+ T cells after antigen stimulation using flow cytometry data. |
| analyze_ebv_antibody_titers | Analyze ELISA data to quantify EBV antibody titers in plasma/serum samples. |
| analyze_cns_lesion_histology | Analyzes histological images of CNS lesions to quantify immune cell infiltration, demyelination, and tissue damage. |
| analyze_immunohistochemistry_image | Analyzes immunohistochemistry images to quantify protein expression and spatial distribution. |

**Table 12:** Tool descriptions for `biomni.microbiology`

| Tool Name | Tool Description |
|---|---|
| optimize_anaerobic_digestion_process | Optimize anaerobic digestion process conditions to maximize VFA production or methane yield. |
| analyze_arsenic_speciation_hplc_icpms | Analyzes arsenic speciation in liquid samples using HPLC-ICP-MS technique and returns a detailed research log. |
| count_bacterial_colonies | Count bacterial colonies from an image of agar plate using computer vision techniques. |
| annotate_bacterial_genome | Annotate a bacterial genome using Prokka to identify genes, proteins, and functional features. |
| enumerate_bacterial_cfu_by_serial_dilution | Quantify bacterial concentration (CFU/mL) using serial dilutions and spot plating. |
| model_bacterial_growth_dynamics | Model bacterial population dynamics over time using ordinary differential equations. |
| quantify_biofilm_biomass_crystal_violet | Quantifies biofilm biomass using crystal violet staining assay data and generates a detailed research log of the analysis. |
| segment_and_analyze_microbial_cells | Perform automated cell segmentation and quantify morphological metrics from fluorescence microscopy images. |
| segment_cells_with_deep_learning | Perform cell segmentation on fluorescence microscopy images using deep learning models from the Cellpose/Omnipose library. |
| simulate_generalized_lotka_volterra_dynamics | Simulate microbial community dynamics using the Generalized Lotka-Volterra (gLV) model. |
| predict_rna_secondary_structure | Predict the secondary structure of an RNA molecule using ViennaRNA and generate visualization files. |
| simulate_microbial_population_dynamics | Performs stochastic simulation of microbial population dynamics using the Gillespie algorithm. |

**Table 13:** Tool descriptions for `biomni.pathology`

| Tool Name | Tool Description |
|---|---|
| `analyze_aortic_diameter_and_geometry` | Analyze aortic diameter and geometry from cardiovascular imaging data to measure aortic root diameter, ascending aorta diameter, and calculate geometric parameters such as tortuosity and dilation indices. |
| `analyze_atp_luminescence_assay` | Analyze luminescence-based ATP assay data to determine intracellular ATP concentration and generate a detailed research log of the analysis. |
| `analyze_thrombus_histology` | Analyze histological images of thrombus samples stained with H&E to identify and quantify different thrombus components (fresh, cellular lysis, endothelialization, fibroblastic reaction). |
| `analyze_intracellular_calcium_with_rhod2` | Analyzes intracellular calcium concentration using Rhod-2 fluorescent indicator from microscopy images. |
| `quantify_corneal_nerve_fibers` | Quantify the volume/density of immunofluorescence-labeled corneal nerve fibers from microscopy images. |
| `segment_and_quantify_cells_in_multiplexed_images` | Segment cells and quantify protein expression levels from multichannel tissue images. |
| `analyze_bone_microct_morphometry` | Analyze bone microarchitecture parameters from 3D micro-CT images, calculating metrics such as bone mineral density (BMD), bone volume (BV), trabecular number (Tb.N), trabecular thickness (Tb.Th), and trabecular separation (Tb.S). |

**Table 14:** Tool descriptions for `biomni.pharmacology`

| Tool Name | Tool Description |
|---|---|
| run_diffdock_with_smiles | Run DiffDock molecular docking simulation using a protein structure and a SMILES string for the ligand. Uses Docker to execute the DiffDock algorithm. |
| docking_autodock_vina | Performs molecular docking using AutoDock Vina to predict binding affinities between small molecules and a receptor protein. |
| run_autosite | Runs AutoSite on a protein structure to identify potential binding sites and returns a research log with the results. |
| retrieve_topk_repurposing_drugs_from_disease_txgnn | Computes TxGNN model predictions for drug repurposing for a given disease and returns the top predicted drugs with their scores. |
| predict_admet_properties | Predicts ADMET (Absorption, Distribution, Metabolism, Excretion, Toxicity) properties for a list of compounds using pretrained models. |
| predict_binding_affinity_protein_1d_sequence | Predicts the binding affinity between small molecules and a protein sequence using pre-trained deep learning models. |
| analyze_accelerated_stability_of_pharmaceutical_formulations | Analyzes the stability of pharmaceutical formulations under accelerated storage conditions and generates a research log of the results. |
| run_3d_chondrogenic_aggregate_assay | Generates a detailed protocol for performing a 3D chondrogenic aggregate culture assay to evaluate compounds' effects on chondrogenesis. |
| grade_adverse_events_using_vcog_ctcae | Grade and monitor adverse events in animal studies using the VCOG-CTCAE standard. |
| analyze_radiolabeled_antibody_biodistribution | Analyze biodistribution and pharmacokinetic profile of radiolabeled antibodies, including tissue distribution, half-lives, and tumor-to-normal tissue ratios. |
| estimate_alpha_particle_radiotherapy_dosimetry | Estimate radiation absorbed doses to tumor and normal organs for alpha-particle radiotherapeutics using the Medical Internal Radiation Dose (MIRD) schema. |
| perform_mwas_cyp2c19_metabolizer_status | Perform a Methylome-wide Association Study (MWAS) to identify CpG sites significantly associated with CYP2C19 metabolizer status. |
| calculate_physicochemical_properties | Calculate key physicochemical properties of a drug candidate molecule including molecular weight, cLogP, TPSA, H-bond donors/acceptors, and other drug-like characteristics. |
| analyze_xenograft_tumor_growth_inhibition | Analyze tumor growth inhibition in xenograft models across different treatment groups. |
| analyze_western_blot | Performs densitometric analysis of Western blot images to quantify relative protein expression. |

**Table 15:** Tool descriptions for `biomni.physiology`

| Tool Name | Tool Description |
|---|---|
| reconstruct_3d_face_from_mri | Generate a 3D model of facial anatomy from MRI scans of the head and neck. |
| analyze_abr_waveform_p1_metrics | Extracts P1 amplitude and latency from Auditory Brainstem Response (ABR) waveform data. P1 (Wave I) is typically the first positive peak in the ABR waveform and is a critical marker for auditory function assessment. |
| analyze_ciliary_beat_frequency | Analyze ciliary beat frequency from high-speed video microscopy data using FFT analysis. |
| analyze_protein_colocalization | Analyze colocalization between two fluorescently labeled proteins in microscopy images. |
| perform_cosinor_analysis | Performs cosinor analysis on physiological time series data to characterize circadian rhythms. |
| calculate_brain_adc_map | Calculate Apparent Diffusion Coefficient (ADC) map from diffusion-weighted MRI data using the monoexponential diffusion model. |
| analyze_endolysosomal_calcium_dynamics | Analyze calcium dynamics in endo-lysosomal compartments using ELGA/ELGA1 probe data. |
| analyze_fatty_acid_composition_by_gc | Analyzes fatty acid composition in tissue samples using gas chromatography data. |
| analyze_hemodynamic_data | Analyzes raw blood pressure data to calculate key hemodynamic parameters including systolic blood pressure, diastolic blood pressure, mean arterial pressure, and heart rate. |
| simulate_thyroid_hormone_pharmacokinetics | Simulates the transport and binding of thyroid hormones across different tissue compartments using an ODE-based pharmacokinetic model. |
| quantify_amyloid_beta_plaques | Analyzes an image to detect, quantify, and characterize amyloid-beta plaques commonly found in Alzheimer's disease tissue samples. |

**Table 16:** Tool descriptions for `biomni.synthetic_biology`

| Tool Name | Tool Description |
|---|---|
| engineer_bacterial_genome_for_therapeutic_delivery | Engineer a bacterial genome by integrating therapeutic genetic parts for therapeutic delivery. |
| analyze_bacterial_growth_rate | Analyze bacterial growth data and extract growth parameters from OD600 measurements. |
| analyze_barcode_sequencing_data | Analyze sequencing data to extract, quantify and determine lineage relationships of barcodes. |
| analyze_bifurcation_diagram | Performs bifurcation analysis on a dynamical system and generates a bifurcation diagram. |
| create_biochemical_network_sbml_model | Generate a mathematical model of a biochemical network in SBML format. |
| optimize_codons_for_heterologous_expression | Analyzes and optimizes a DNA/RNA sequence for improved expression in a heterologous host organism. |
| simulate_gene_circuit_with_growth_feedback | Simulate gene regulatory circuit dynamics with growth feedback, tracking gene expression levels and cell growth over time. |
| identify_fas_functional_domains | Identifies functional domains within a Fatty Acid Synthase (FAS) sequence and predicts their roles. |

**Table 17:** Tool descriptions for `biomni.systems_biology`

| Tool Name | Tool Description |
|---|---|
| perform_flux_balance_analysis | Perform Flux Balance Analysis (FBA) on a genome-scale metabolic network model to predict metabolic flux distributions. |
| model_protein_dimerization_network | Model protein dimerization networks to find equilibrium concentrations of dimers based on monomer concentrations and binding affinities. |
| simulate_metabolic_network_perturbation | Construct and simulate kinetic models of metabolic networks and analyze their responses to perturbations. |
| simulate_protein_signaling_network | Simulate protein signaling network dynamics using ODE-based logic modeling with normalized Hill functions. |
| compare_protein_structures | Compares two protein structures to identify structural differences and conformational changes. |
| simulate_renin_angiotensin_system_dynamics | Simulate the time-dependent concentrations of renin-angiotensin system (RAS) components. |

**Table 18:** Tool descriptions for `biomni.support_tools`

| Tool Name | Tool Description |
|---|---|
| run_python_repl | Executes a Python command in the notebook environment and returns the output as a string. |
| read_function_source_code | Read the source code of a function from any module path. |

**Table 19:** Tool descriptions for `biomni.database`

| Tool Name | Tool Description |
|---|---|
| query_uniprot | Query the UniProt REST API using either natural language or a direct endpoint to retrieve protein information. |
| query_alphafold | Query the AlphaFold Database API for protein structure predictions and information. |
| query_interpro | Query the InterPro REST API using natural language or a direct endpoint to retrieve information about protein domains or families. |
| query_pdb | Query the RCSB PDB database using natural language or a direct structured query to find protein structures. |
| query_pdb_identifiers | Retrieve detailed data and/or download files for PDB identifiers. |
| query_kegg | Take a natural language prompt and convert it to a structured KEGG API query, then execute the query. |
| query_stringdb | Query the STRING protein interaction database using natural language or direct endpoint. |
| query_iucn | Query the IUCN Red List API using natural language or a direct endpoint to retrieve species conservation status information. |
| query_paleobiology | Query the Paleobiology Database (PBDB) API using natural language or a direct endpoint. |
| query_jaspar | Query the JASPAR REST API using natural language or a direct endpoint to retrieve transcription factor binding profiles. |
| query_worms | Query the World Register of Marine Species (WoRMS) REST API using natural language or a direct endpoint. |
| query_cbioportal | Query the cBioPortal REST API using natural language or a direct endpoint to access cancer genomics data. |
| query_clinvar | Take a natural language prompt and convert it to a structured ClinVar query to search for genetic variants. |
| query_geo | Query the NCBI Gene Expression Omnibus (GEO) using natural language or a direct search term. |
| query_dbsnp | Query the NCBI dbSNP database using natural language or a direct search term. |
| query_ucsc | Query the UCSC Genome Browser API using natural language or a direct endpoint. |

**Table 20:** Continue tool descriptions for `biomni.database`

| Tool Name | Tool Description |
|---|---|
| query_ensembl | Query the Ensembl REST API using natural language or a direct endpoint to retrieve genomic data. |
| query_opentarget_genetics | Query the OpenTargets Genetics API using natural language or a direct GraphQL query to retrieve information about genetic targets and variants. |
| query_opentarget | Query the OpenTargets Platform API using natural language or a direct GraphQL query to access drug targets, diseases, and mechanisms data. |
| query_gwas_catalog | Query the GWAS Catalog API using natural language or a direct endpoint to retrieve genetic association studies data. |
| query_gnomad | Query gnomAD for variants in a gene using natural language or direct gene symbol. |
| blast_sequence | Identifies a DNA or protein sequence using NCBI BLAST and returns information about the best alignment. |
| query_reactome | Query the Reactome database using natural language or a direct endpoint to retrieve information about biological pathways. |
| query_regulomedb | Query the RegulomeDB database using natural language or direct endpoint specification to get information about regulatory elements. |
| query_pride | Query the PRIDE (PRoteomics IDEntifications) database using natural language or a direct endpoint. |
| query_gtopdb | Query the Guide to PHARMACOLOGY database (GtoPdb) using natural language or a direct endpoint. |
| region_to_ccre_screen | Retrieves information about candidate cis-regulatory elements (cCREs) that intersect with a specified genomic region. |
| get_genes_near_ccre | Identifies the nearest genes to a specified candidate cis-Regulatory Element (cCRE) by querying the SCREEN database. |
| query_remap | Query the ReMap database for regulatory elements and transcription factor binding sites using natural language or direct API endpoints. |
| query_mpd | Query the Mouse Phenome Database (MPD) for mouse strain phenotype data using natural language or direct endpoint access. |
| query_emdb | Query the Electron Microscopy Data Bank (EMDB) for 3D macromolecular structures using natural language or direct endpoint access. |

**Table 21:** Data lake descriptions

| File Name | Description |
|---|---|
| affinity_capture-ms.csv | Protein-protein interactions detected via affinity capture and mass spectrometry. |
| affinity_capture-rna.csv | Protein-RNA interactions detected by affinity capture. |
| BindingDB_All_202409.tsv | Measured binding affinities between proteins and small molecules for drug discovery. |
| broad_repurposing_hub_molecule_with_smiles.csv | Molecules from Broad Institute's Drug Repurposing Hub with SMILES annotations. |
| broad_repurposing_hub_phase_moa_target_info.csv | Drug phases, mechanisms of action, and target information from Broad Institute. |
| co-fractionation.csv | Protein-protein interactions from co-fractionation experiments. |
| Cosmic_Breakpoints_v101_GRCh38.csv | Genomic breakpoints associated with cancers from COSMIC database. |
| Cosmic_CancerGeneCensusHallmarksOfCancer_v101_GRCh38.csv | Hallmarks of cancer genes from COSMIC. |
| Cosmic_CancerGeneCensus_v101_GRCh38.csv | Census of cancer-related genes from COSMIC. |
| Cosmic_ClassificationPaper_v101_GRCh38.csv | Cancer classifications and annotations from COSMIC. |
| Cosmic_Classification_v101_GRCh38.csv | Classification of cancer types from COSMIC. |
| Cosmic_CompleteCNA_v101_GRCh38.tsv.gz | Complete copy number alterations data from COSMIC. |
| Cosmic_CompleteDifferentialMethylation_v101_GRCh38.tsv.gz | Differential methylation patterns from COSMIC. |
| Cosmic_CompleteGeneExpression_v101_GRCh38.tsv.gz | Gene expression data across cancers from COSMIC. |
| Cosmic_Fusion_v101_GRCh38.csv | Gene fusion events from COSMIC. |
| Cosmic_Genes_v101_GRCh38.csv | List of genes associated with cancer from COSMIC. |
| Cosmic_GenomeScreensMutant_v101_GRCh38.tsv.gz | Genome screening mutations from COSMIC. |
| Cosmic_MutantCensus_v101_GRCh38.csv | Catalog of cancer-related mutations from COSMIC. |
| Cosmic_ResistanceMutations_v101_GRCh38.csv | Resistance mutations related to therapeutic interventions from COSMIC. |
| czi_census_datasets_v4.csv | Datasets from the Chan Zuckerberg Initiative's Cell Census. |
| DisGeNET.csv | Gene-disease associations from multiple sources. |
| dosage_growth_defect.csv | Gene dosage changes affecting growth. |
| enamine_cloud_library_smiles.pkl | Compounds from Enamine REAL library with SMILES annotations. |
| genebass_missense_LC_filtered.pkl | Filtered missense variants from GeneBass. |
| genebass_pLoF_filtered.pkl | Predicted loss-of-function variants from GeneBass. |
| genebass_synonymous_filtered.pkl | Filtered synonymous variants from GeneBass. |
| gene_info.csv | Comprehensive gene information. |
| genetic_interaction.csv | Genetic interactions between genes. |
| go-plus.json | Gene ontology data for functional gene annotations. |
| gtex_tissue_gene_tpm.csv | Gene expression (TPM) across human tissues from GTEx. |
| gwas_catalog.pkl | Genome-wide association studies (GWAS) results. |
| marker_celltype.csv | Cell type marker genes for identification. |
| McPAS-TCR.csv | T-cell receptor sequences and specificity data from McPAS database. |
| miRDB_v6.0_results.csv | Predicted microRNA targets from miRDB. |
| miRTarBase_microRNA_target_interaction.csv | Experimentally validated microRNA-target interactions from miRTarBase. |
| miRTarBase_microRNA_target_interaction_pubmed_abtract.txt | PubMed abstracts for microRNA-target interactions in miRTarBase. |
| miRTarBase_MicroRNA_Target_Sites.csv | Binding sites of microRNAs on target genes from miRTarBase. |

**Table 22:** Continue data lake descriptions

| File Name | Description |
|---|---|
| mousemine_m1_positional_geneset.csv | Positional gene sets from MouseMine. |
| mousemine_m2_curated_geneset.csv | Curated gene sets from MouseMine. |
| mousemine_m3_regulatory_target_geneset.csv | Regulatory target gene sets from MouseMine. |
| mousemine_m5_ontology_geneset.csv | Ontology-based gene sets from MouseMine. |
| mousemine_m8_celltype_signature_geneset.csv | Cell type signature gene sets from MouseMine. |
| mousemine_mh_hallmark_geneset.csv | Hallmark gene sets from MouseMine. |
| msigdb_human_c1_positional_geneset.csv | Human positional gene sets from MSigDB. |
| msigdb_human_c2_curated_geneset.csv | Curated human gene sets from MSigDB. |
| msigdb_human_c3_regulatory_target_geneset.csv | Regulatory target gene sets from MSigDB. |
| msigdb_human_c3_subset_transcription_factor_targets_from_GTRD.csv | Transcription factor targets from GTRD/MSigDB. |
| msigdb_human_c4_computational_geneset.csv | Computationally derived gene sets from MSigDB. |
| msigdb_human_c5_ontology_geneset.csv | Ontology-based gene sets from MSigDB. |
| msigdb_human_c6_oncogenic_signature_geneset.csv | Oncogenic signatures from MSigDB. |
| msigdb_human_c7_immunologic_signature_geneset.csv | Immunologic signatures from MSigDB. |
| msigdb_human_c8_celltype_signature_geneset.csv | Cell type signatures from MSigDB. |
| msigdb_human_h_hallmark_geneset.csv | Hallmark gene sets from MSigDB. |
| omim.csv | Genetic disorders and associated genes from OMIM. |
| proteinatlas.tsv | Protein expression data from Human Protein Atlas. |
| proximity_label-ms.csv | Protein interactions via proximity labeling and mass spectrometry. |
| reconstituted_complex.csv | Protein complexes reconstituted in vitro. |
| synthetic_growth_defect.csv | Synthetic growth defects from genetic interactions. |
| synthetic_lethality.csv | Synthetic lethal interactions. |
| synthetic_rescue.csv | Genetic interactions rescuing phenotypes. |
| two-hybrid.csv | Protein-protein interactions detected by yeast two-hybrid assays. |
| variant_table.csv | Annotated genetic variants table. |
| Virus-Host_PPI_P-HIPSTER_2020.csv | Virus-host protein-protein interactions from P-HIPSTER. |

**Table 23:** Software descriptions

| Software Name | Description |
|---|---|
| `biopython` | [Python Package] A set of tools for biological computation including parsers for bioinformatics files, access to online services, and interfaces to common bioinformatics programs. |
| `biom-format` | [Python Package] The Biological Observation Matrix (BIOM) format is designed for representing biological sample by observation contingency tables with associated metadata. |
| `scanpy` | [Python Package] A scalable toolkit for analyzing single-cell gene expression data, specifically designed for large datasets using AnnData. |
| `scikit-bio` | [Python Package] Data structures, algorithms, and educational resources for bioinformatics, including sequence analysis, phylogenetics, and ordination methods. |
| `anndata` | [Python Package] A Python package for handling annotated data matrices in memory and on disk, primarily used for single-cell genomics data. |
| `mudata` | [Python Package] A Python package for multimodal data storage and manipulation, extending AnnData to handle multiple modalities. |
| `pyliftover` | [Python Package] A Python implementation of UCSC liftOver tool for converting genomic coordinates between genome assemblies. |
| `biopandas` | [Python Package] A package that provides pandas DataFrames for working with molecular structures and biological data. |
| `biotite` | [Python Package] A comprehensive library for computational molecular biology, providing tools for sequence analysis, structure analysis, and more. |
| `gget` | [Python Package] A toolkit for accessing genomic databases and retrieving sequences, annotations, and other genomic data. |
| `lifelines` | [Python Package] A complete survival analysis library for fitting models, plotting, and statistical tests. |
| `scvi-tools` | [Python Package] A package for probabilistic modeling of single-cell omics data, including deep generative models. |

**Table 24:** Continue software descriptions

| Software Name | Description |
|---|---|
| `gseapy` | [Python Package] A Python wrapper for Gene Set Enrichment Analysis (GSEA) and visualization. |
| `scrublet` | [Python Package] A tool for detecting doublets in single-cell RNA-seq data. |
| `cellxgene-census` | [Python Package] A tool for accessing and analyzing the CellxGene Census, a collection of single-cell datasets. |
| `hyperopt` | [Python Package] A Python library for optimizing hyperparameters of machine learning algorithms. |
| `scvelo` | [Python Package] A tool for RNA velocity analysis in single cells using dynamical models. |
| `pysam` | [Python Package] A Python module for reading, manipulating and writing genomic data sets in SAM/BAM/VCF/BCF formats. |
| `pyfaidx` | [Python Package] A Python package for efficient random access to FASTA files. |
| `pyranges` | [Python Package] A Python package for interval manipulation with a pandas-like interface. |
| `pybedtools` | [Python Package] A Python wrapper for Aaron Quinlan's BEDTools programs. |
| `rdkit` | [Python Package] A collection of cheminformatics and machine learning tools for working with chemical structures and drug discovery. |
| `deeppurpose` | [Python Package] A deep learning library for drug-target interaction prediction and virtual screening. |
| `pyscreener` | [Python Package] A Python package for virtual screening of chemical compounds. |
| `openbabel` | [Python Package] A chemical toolbox designed to speak the many languages of chemical data, supporting file format conversion and molecular modeling. |
| `descriptastorus` | [Python Package] A library for computing molecular descriptors for machine learning applications in drug discovery. |

**Table 25:** Continue software descriptions

| Software Name | Description |
|---|---|
| `pymol` | [Python Package] A molecular visualization system for rendering and animating 3D molecular structures. |
| `openmm` | [Python Package] A toolkit for molecular simulation using high-performance GPU computing. |
| `pytdc` | [Python Package] A Python package for Therapeutics Data Commons, providing access to machine learning datasets for drug discovery. |
| `pandas` | [Python Package] A fast, powerful, and flexible data analysis and manipulation library for Python. |
| `numpy` | [Python Package] The fundamental package for scientific computing with Python, providing support for arrays, matrices, and mathematical functions. |
| `scipy` | [Python Package] A Python library for scientific and technical computing, including modules for optimization, linear algebra, integration, and statistics. |
| `scikit-learn` | [Python Package] A machine learning library featuring various classification, regression, and clustering algorithms. |
| `matplotlib` | [Python Package] A comprehensive library for creating static, animated, and interactive visualizations in Python. |
| `seaborn` | [Python Package] A statistical data visualization library based on matplotlib with a high-level interface for drawing attractive statistical graphics. |
| `statsmodels` | [Python Package] A Python module for statistical modeling and econometrics, including descriptive statistics and estimation of statistical models. |
| `pymc3` | [Python Package] A Python package for Bayesian statistical modeling and probabilistic machine learning. |
| `pystan` | [Python Package] A Python interface to Stan, a platform for statistical modeling and high-performance statistical computation. |
| `umap-learn` | [Python Package] Uniform Manifold Approximation and Projection, a dimension reduction technique. |

**Table 26:** Continue software descriptions

| Software Name | Description |
|---|---|
| faiss-cpu | [Python Package] A library for efficient similarity search and clustering of dense vectors. |
| harmony-pytorch | [Python Package] A PyTorch implementation of the Harmony algorithm for integrating single-cell data. |
| tiledb | [Python Package] A powerful engine for storing and analyzing large-scale genomic data. |
| tiledbsoma | [Python Package] A library for working with the SOMA (Stack of Matrices) format using TileDB. |
| h5py | [Python Package] A Python interface to the HDF5 binary data format, allowing storage of large amounts of numerical data. |
| tqdm | [Python Package] A fast, extensible progress bar for loops and CLI applications. |
| joblib | [Python Package] A set of tools to provide lightweight pipelining in Python, including transparent disk-caching and parallel computing. |
| opencv-python | [Python Package] OpenCV library for computer vision tasks, useful for image analysis in biological contexts. |
| PyPDF2 | [Python Package] A library for working with PDF files, useful for extracting text from scientific papers. |
| googlesearch-python | [Python Package] A library for performing Google searches programmatically. |
| scikit-image | [Python Package] A collection of algorithms for image processing in Python. |
| pymed | [Python Package] A Python library for accessing PubMed articles. |
| arxiv | [Python Package] A Python wrapper for the arXiv API, allowing access to scientific papers. |

**Table 27:** Continue software descriptions

| Software Name | Description |
|---|---|
| `scholarly` | [Python Package] A module to retrieve author and publication information from Google Scholar. |
| `cryosparc-tools` | [Python Package] Tools for working with cryoSPARC, a platform for cryo-EM data processing. |
| `mageck` | [Python Package] Analysis of CRISPR screen data. |
| `igraph` | [Python Package] Network analysis and visualization. |
| `pyscenic` | [Python Package] Analysis of single-cell RNA-seq data and gene regulatory networks. |
| `cooler` | [Python Package] Storage and analysis of Hi-C data. |
| `trackpy` | [Python Package] Particle tracking in images and video. |
| `flowcytometrytools` | [Python Package] Analysis and visualization of flow cytometry data. |
| `cellpose` | [Python Package] Cell segmentation in microscopy images. |
| `viennarna` | [Python Package] RNA secondary structure prediction. |
| `PyMassSpec` | [Python Package] Mass spectrometry data analysis. |
| `python-libsbml` | [Python Package] Working with SBML files for computational biology. |
| `cobra` | [Python Package] Constraint-based modeling of metabolic networks. |
| `reportlab` | [Python Package] Creation of PDF documents. |
| `flowkit` | [Python Package] Toolkit for processing flow cytometry data. |
| `hmmlearn` | [Python Package] Hidden Markov model analysis. |
| `msprime` | [Python Package] Simulation of genetic variation. |
| `tskit` | [Python Package] Handling tree sequences and population genetics data. |
| `cyvcf2` | [Python Package] Fast parsing of VCF files. |
| `pykalman` | [Python Package] Kalman filter and smoother implementation. |

**Table 28:** Continue software descriptions

| Software Name | Description |
|---|---|
| `fanc` | [Python Package] Analysis of chromatin conformation data. |
| `ggplot2` | [R Package] A system for declaratively creating graphics, based on The Grammar of Graphics. Use with subprocess.run(['Rscript', '-e', 'library(ggplot2); ...']). |
| `dplyr` | [R Package] A grammar of data manipulation, providing a consistent set of verbs that help you solve the most common data manipulation challenges. Use with subprocess. |
| `tidyr` | [R Package] A package that helps you create tidy data, where each column is a variable, each row is an observation, and each cell is a single value. Use with subprocess. |
| `readr` | [R Package] A fast and friendly way to read rectangular data like CSV, TSV, and FWF. Use with subprocess.run(['Rscript', '-e', 'library(readr); ...']). |
| `stringr` | [R Package] A cohesive set of functions designed to make working with strings as easy as possible. Use with subprocess calls. |
| `Matrix` | [R Package] A package that provides classes and methods for dense and sparse matrices. Required for Seurat. Use with subprocess calls. |
| `Rcpp` | [R Package] Seamless R and C++ Integration, allowing R functions to call compiled C++ code. Use with subprocess calls. |
| `devtools` | [R Package] Tools to make developing R packages easier, including functions to install packages from GitHub. Use with subprocess calls. |
| `remotes` | [R Package] Install R packages from GitHub, GitLab, Bitbucket, or other remote repositories. Use with subprocess calls. |
| `DESeq2` | [R Package] Differential gene expression analysis based on the negative binomial distribution. Use with subprocess.run(['Rscript', '-e', 'library(DESeq2); ...']). |

**Table 29:** Continue software descriptions

| Software Name | Description |
|---|---|
| `clusterProfiler` | [R Package] A package for statistical analysis and visualization of functional profiles for genes and gene clusters. Use with subprocess calls. |
| `DADA2` | [R Package] A package for modeling and correcting Illumina-sequenced amplicon errors. Use with subprocess calls. |
| `xcms` | [R Package] A package for processing and visualization of LC-MS and GC-MS data. Use with subprocess calls. |
| `FlowCore` | [R Package] Basic infrastructure for flow cytometry data. Use with subprocess calls. |
| `edgeR` | [R Package] Empirical Analysis of Digital Gene Expression Data in R, for differential expression analysis. Use with subprocess calls. |
| `limma` | [R Package] Linear Models for Microarray Data, for differential expression analysis. Use with subprocess calls. |
| `harmony` | [R Package] A method for integrating and analyzing single-cell data across datasets. Use with subprocess calls. |
| `WGCNA` | [R Package] Weighted Correlation Network Analysis for studying biological networks. Use with subprocess calls. |
| `samtools` | [CLI Tool] A suite of programs for interacting with high-throughput sequencing data. Use with subprocess.run(['samtools', ...]). |
| `bowtie2` | [CLI Tool] An ultrafast and memory-efficient tool for aligning sequencing reads to long reference sequences. Use with subprocess.run(['bowtie2', ...]). |
| `bwa` | [CLI Tool] Burrows-Wheeler Aligner for mapping low-divergent sequences against a large reference genome. Use with subprocess.run(['bwa', ...]). |

**Table 30:** Continue software descriptions

| Software Name | Description |
|---|---|
| `bedtools` | [CLI Tool] A powerful toolset for genome arithmetic, allowing operations like intersect, merge, count, and complement on genomic features. Use with subprocess.run(['bedtools', ...]). |
| `macs2` | [CLI Tool] Model-based Analysis of ChIP-Seq data, a tool for identifying transcript factor binding sites. |
| `fastqc` | [CLI Tool] A quality control tool for high throughput sequence data. Use with subprocess.run(['fastqc', ...]). |
| `trimmomatic` | [CLI Tool] A flexible read trimming tool for Illumina NGS data. Use with subprocess.run(['trimmomatic', ...]). |
| `mafft` | [CLI Tool] A multiple sequence alignment program for unix-like operating systems. Use with subprocess.run(['mafft', ...]). |
| `Homer` | [CLI Tool] Motif discovery and next-gen sequencing analysis. |
| `FastTree` | [CLI Tool] Phylogenetic trees from sequence alignments. |
| `muscle` | [CLI Tool] Multiple sequence alignment tool. |
| `plink2` | [CLI Tool] A comprehensive toolkit for genome association studies that can perform a range of large-scale analyses in a computationally efficient manner. Use with subprocess.run(['plink2', ...]). |
| `gcta64` | [CLI Tool] Genome-wide Complex Trait Analysis (GCTA) tool for estimating the proportion of phenotypic variance explained by genome-wide SNPs and analyzing genetic relationships. Use with subprocess.run(['gcta64', ...]). |
| `iqtree2` | [CLI Tool] An efficient phylogenetic software for maximum likelihood analysis with built-in model selection and ultrafast bootstrap. Use with subprocess.run(['iqtree2', ...]). |

**Table 31:** Completeness scoring rubric for experimental documentation.

| Score | Level | Description |
|---|---|---|
| 1 | Severely Incomplete | Missing many major components necessary for experiments. Cannot be followed for implementation. Lacks critical methodological details and equipment specifications. |
| 2 | Significantly Incomplete | Missing few major components. Requires substantial additional effort from user. Key experimental parameters or control measures absent. |
| 3 | Moderately Complete | Contains basic components with some detail gaps. Requires moderate additional effort from user. Most critical parameters defined but needs optimization. |
| 4 | Mostly Complete | Contains nearly all necessary components. Requires minimal additional effort from user. Well-described procedures with adequate troubleshooting guidance. |
| 5 | Completely Thorough | Contains all necessary components in appropriate detail. Can be followed precisely with no additional effort. Comprehensive methodology, specifications, and contingency plans. |

**Table 32:** Accuracy scoring rubric for experimental documentation.

| Score | Level | Description |
|---|---|---|
| 1 | Severely Inaccurate | Contains many major errors or misconceptions. Methods would lead to invalid or unreliable results. Fundamental scientific principles violated or misapplied. |
| 2 | Significantly Inaccurate | Contains few major errors or several minor errors. Some methodological approaches are flawed. May lead to partially valid but questionable results. |
| 3 | Moderately Accurate | Generally correct but contains some minor errors or imprecisions. Core methodology is sound but may have optimization issues. Results would be acceptable but not optimal. |
| 4 | Mostly Accurate | Contains minimal errors. Methods align with accepted scientific practices. Results would be reliable with only minor corrections needed. |
| 5 | Completely Accurate | Contains no errors. Methods perfectly align with best scientific practices. Results would be highly reliable and reproducible. Includes appropriate controls and validation steps. |